

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



Deep Learning

Aspectos prácticos de CNN

Entrenar y luego utilizar de forma práctica las redes CNN no es algo trivial

Para estudiar los aspectos prácticos, revisaremos 3 preguntas:

- ¿Cómo entrenar/optimizar?
- ¿Cómo reutilizar lo ya aprendido por una CNN?
- ¿Cómo mejorar las CNNs para llevarlas a entornos reales?

Entrenar y luego utilizar de forma práctica las redes CNN no es algo trivial

Para estudiar los aspectos prácticos, revisaremos 3 preguntas:

- ¿Cómo entrenar/optimizar?
- ¿Cómo reutilizar lo ya aprendido por una CNN?
- ¿Cómo mejorar las CNNs para llevarlas a entornos reales?

Qué y para qué estamos optimizando

$$\operatorname{argmin}_W J(X, Y; W) = \lambda \mathcal{R}(W) + \sum_i^N \mathcal{L}(x_i, y_i; W)$$



Regularización explícita de los
parámetros



Suma de la función de
pérdida por cada ejemplo

La función de pérdida depende de los puntajes de la red, por lo que es necesario propagar su gradiente

Cross Entropy Loss (aka regresión logística multinomial)

scores = unnormalized log probabilities of the classes.

$$P(Y = k|X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \quad \text{where} \quad s = f(x_i; W)$$

Want to maximize the log likelihood, or (for a loss function) to minimize the negative log likelihood of the correct class:

$$L_i = -\log P(Y = y_i|X = x_i)$$

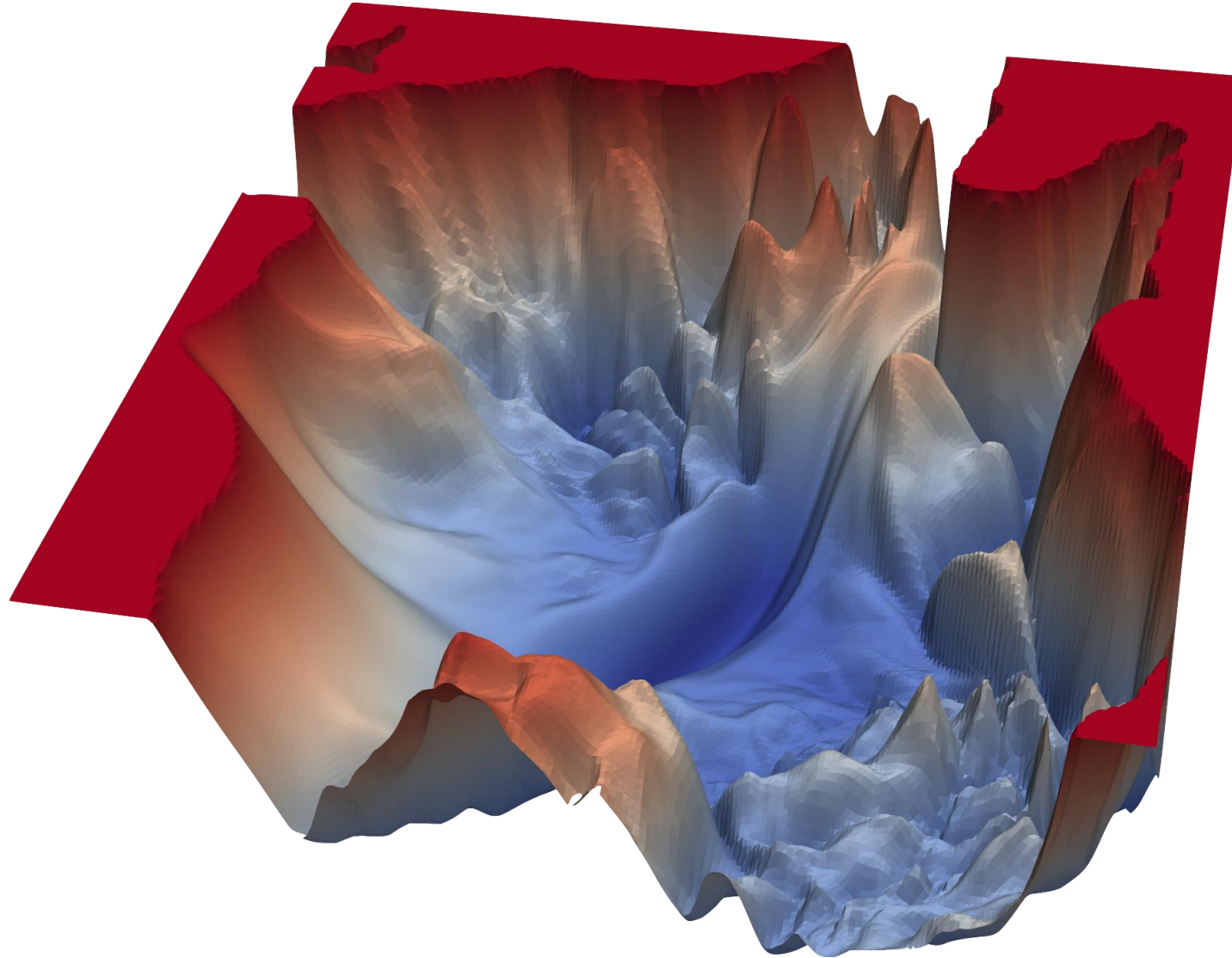
in summary: $L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$

Qué y para qué estamos optimizando

Para obtener los parámetros de una red a través de un proceso de optimización, debemos considerar los siguientes elementos:

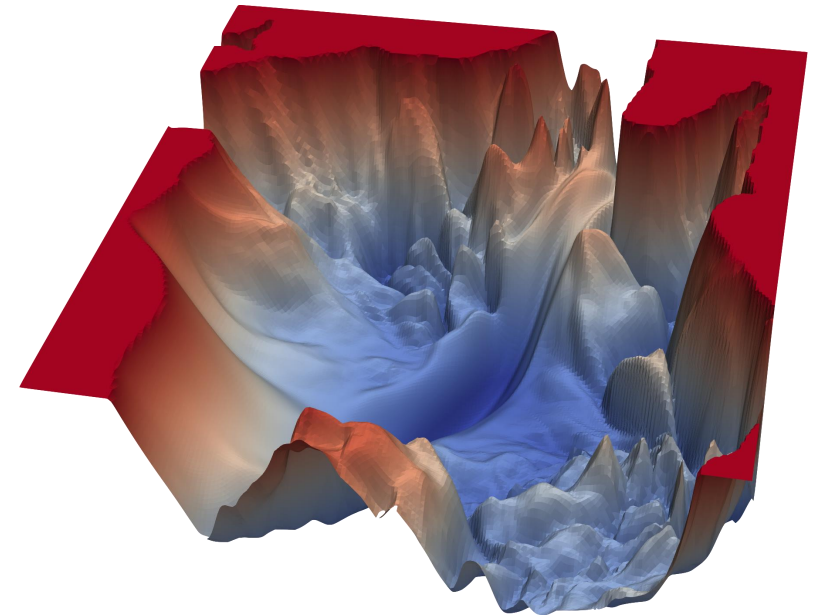
- ¿Cómo calculo las derivadas para cada parámetro?
- ¿Cómo descendo en la función objetivo?

Uno de los motivos por el cual no es suficiente un simple descenso de gradiente



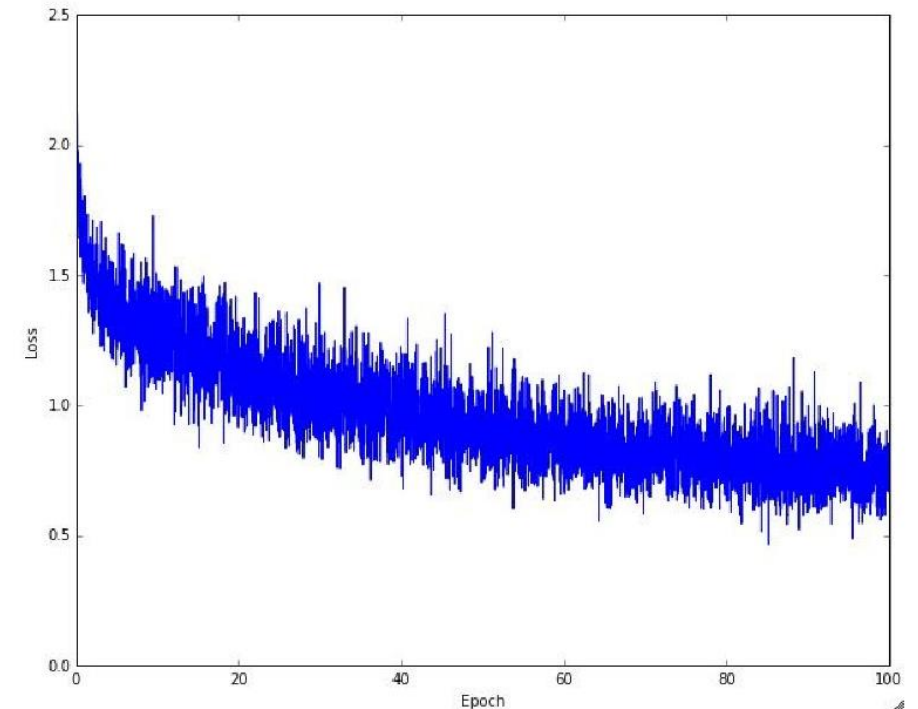
Algunos de los desafíos

- Costo de cómputo del **gradiente completo**
- Escapar de **puntos críticos** y mínimos locales
- Ajuste **dinámico** del learning rate
- **Escala** del learning rate para cada parámetro



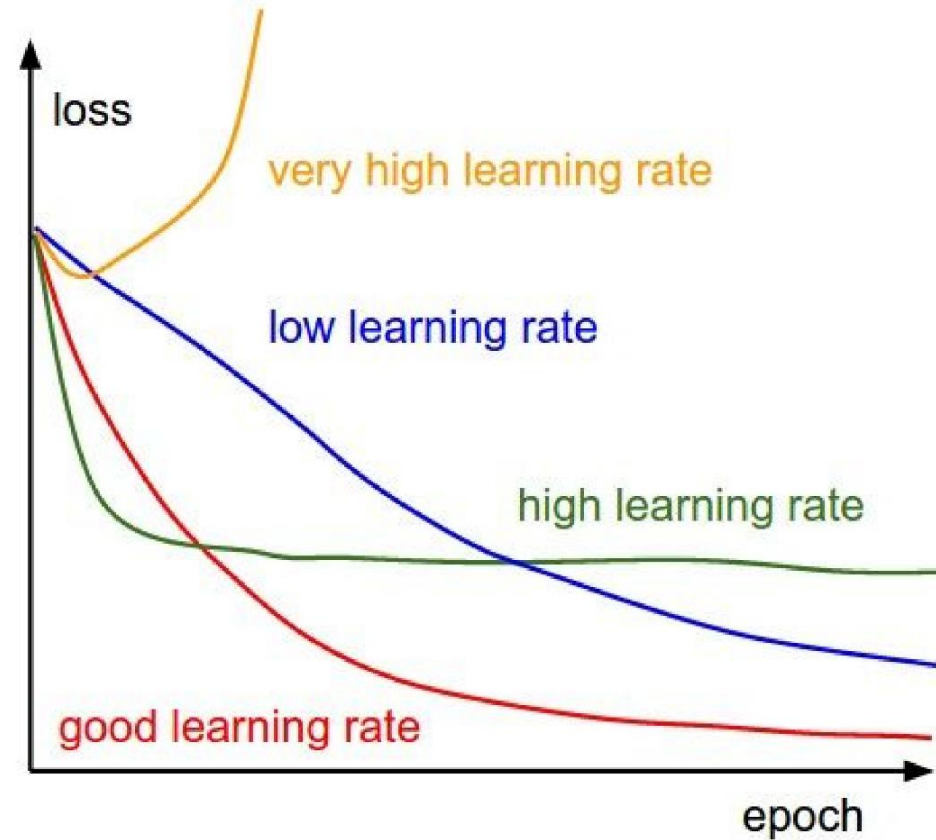
Stochastic Gradient Descent (SGD) permite entrenar modelos usando gran volumen de datos

- Consiste en usar un mini-batch de ejemplos para calcular el gradiente para cada descenso, en vez de todos los ejemplos.
- Existen garantías teóricas de convergencia (tomando un learning rate suficientemente pequeño)
- Tamaños comunes son 32/64/128/256 cuando se utiliza una única GPU.
- Estocasticidad permite evitar ciertos puntos críticos



Stochastic Gradient Descent (SGD) permite entrenar modelos usando gran volumen de datos

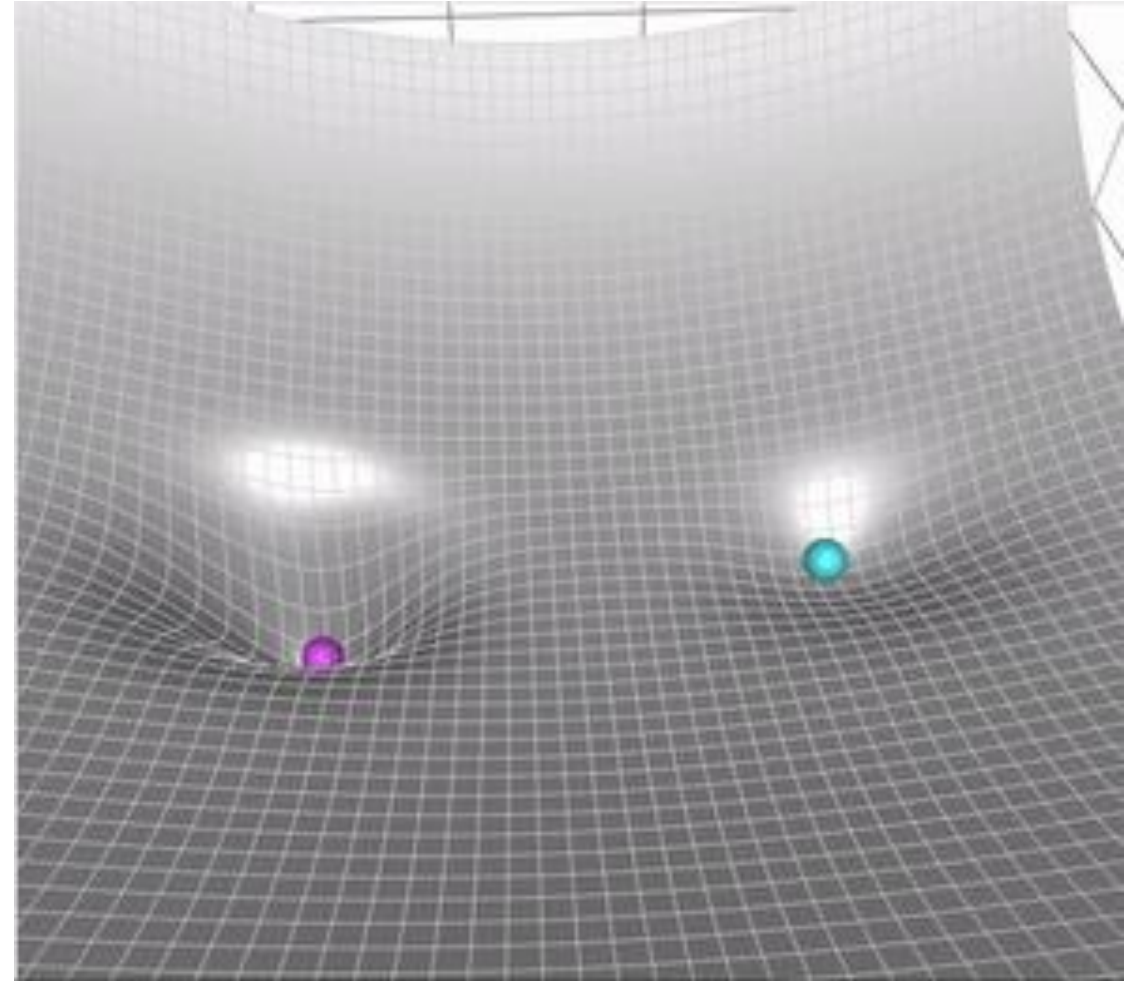
The effects of step size (or “learning rate”)



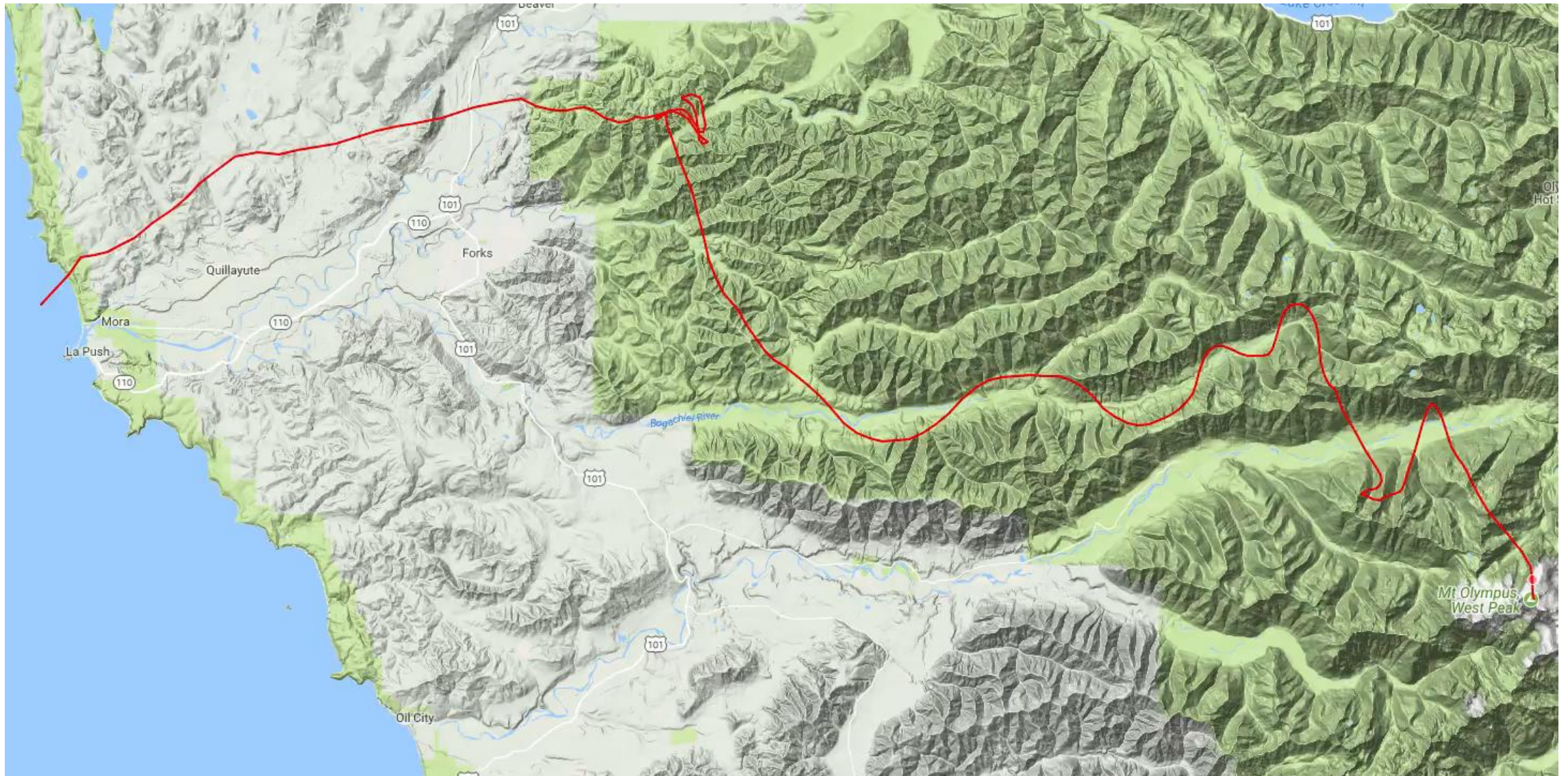
Momentum permite aminorar riesgo de quedar pegado en puntos críticos

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$



Un ejemplo muy “bonito” puede construirse utilizando datos geográficos de elevación



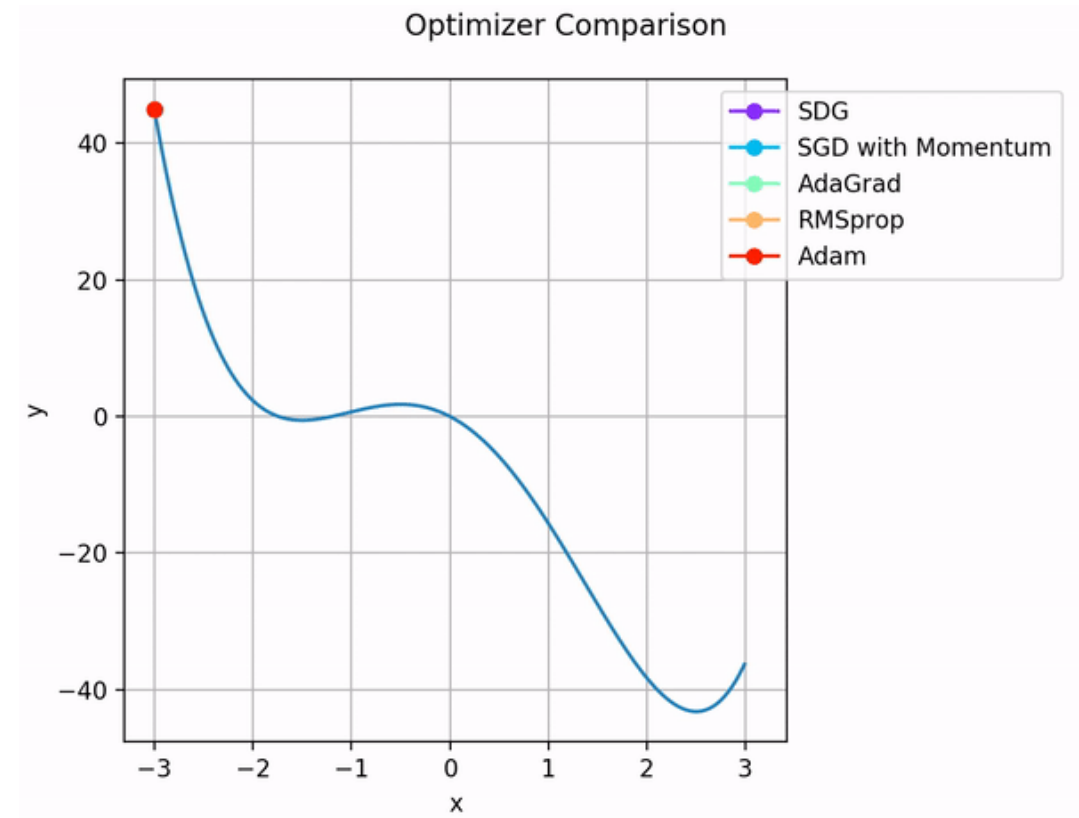
¹<https://fosterelli.co/executing-gradient-descent-on-the-earth>

Adam combina momentum con *learning rate* adaptativo

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \rightarrow \hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \rightarrow \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$



Entrenar y luego utilizar de forma práctica las redes CNN no es algo trivial

Para estudiar los aspectos prácticos, revisaremos 3 preguntas:

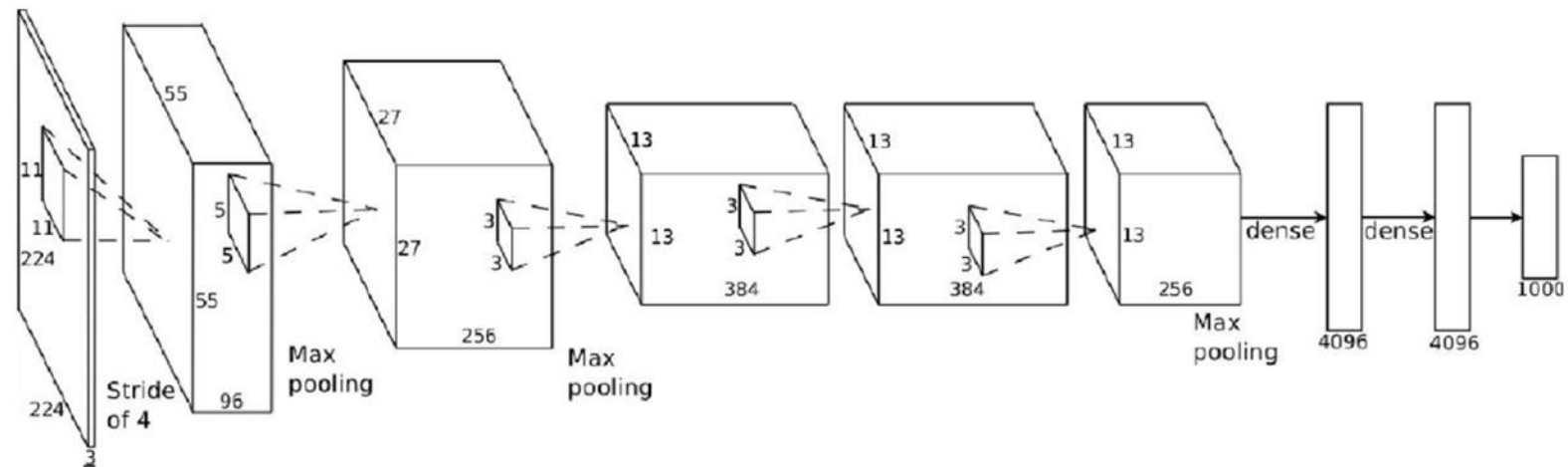
- ¿Cómo entrenar/optimizar?
- ¿Cómo reutilizar lo ya aprendido por una CNN?
- ¿Cómo mejorar las CNNs para llevarlas a entornos reales?

Reutilización es la máxima evidencia de generalización
(y ahorro en costos, a fin de cuentas)

- Resultados de DL con imágenes son claramente impresionantes, pero presentan un problema muy puntual (clasificación de imágenes), que no necesariamente se alinea con muchos casos de uso reales.
- Afortunadamente, y a diferencia de los métodos que hemos estudiado antes, las redes neuronales profundas ya entrenadas pueden ser usadas/adaptadas con poco esfuerzo en otras tareas.
- Esta propiedad es una consecuencia de la habilidad de las redes para aprender *features* jerárquicas y composicionales, que poseen capacidades de generalización altas.

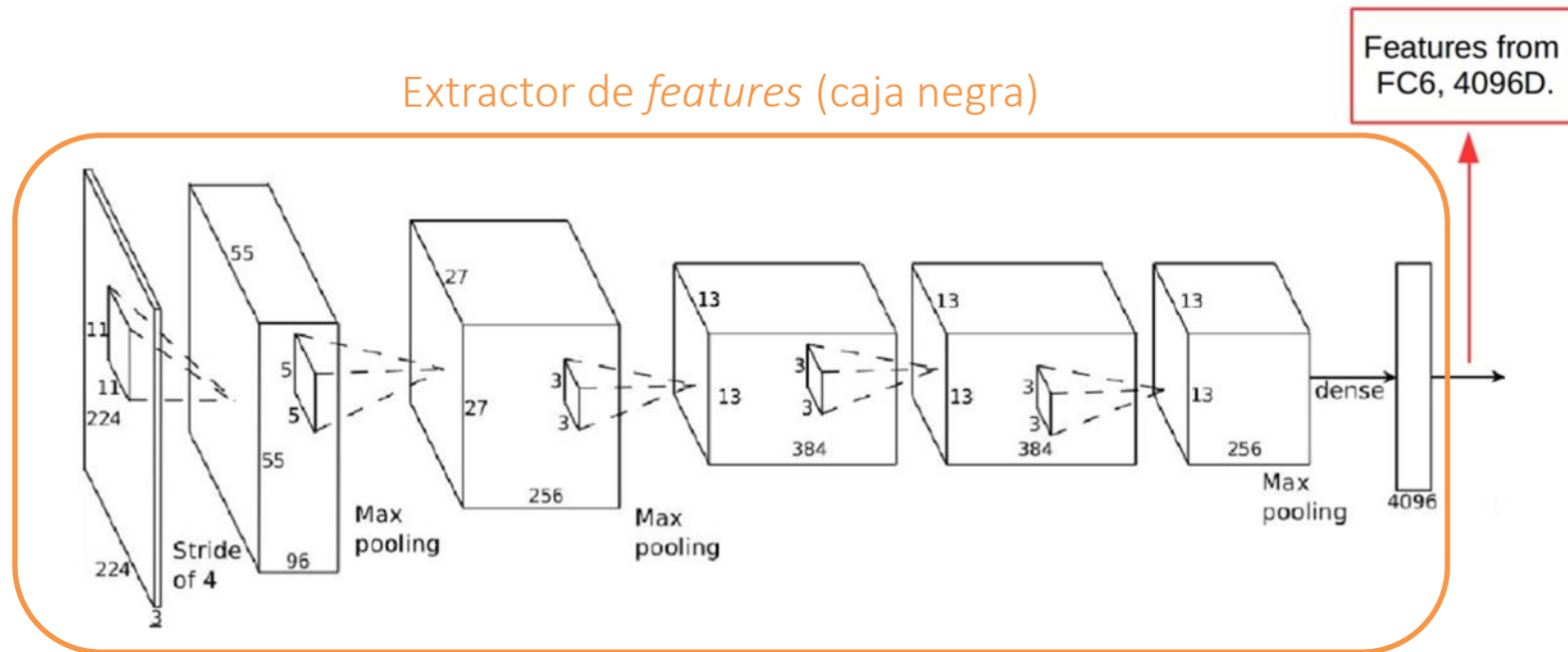
AlexNet nos da un buen ejemplo para entender intuitivamente la reutilización

Si le “cortamos la cabeza” y la utilizamos como extractor de características, se obtienen excelentes resultados en una gran cantidad de tareas visuales: reconocimiento de escenas, caras, acciones, personas, etc.



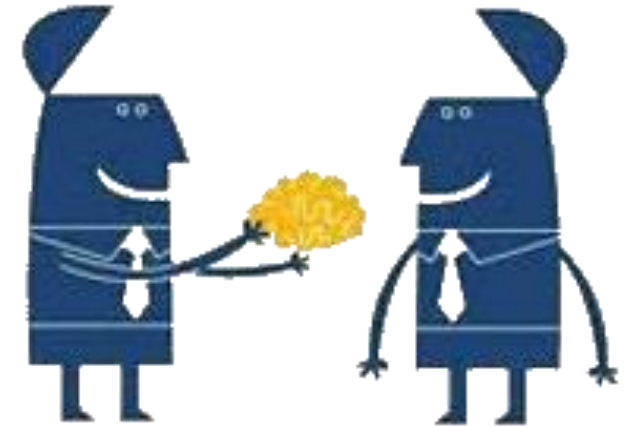
AlexNet nos da un buen ejemplo para entender intuitivamente la reutilización

Si le “cortamos la cabeza” y la utilizamos como extractor de características, se obtienen excelentes resultados en una gran cantidad de tareas visuales: reconocimiento de escenas, caras, acciones, personas, etc.



Este tipo de ideas son parte de un área llamada *Transfer Learning*

- En resumen, la idea es aprender inicialmente modelos poderosos utilizando conjuntos de datos abundantes y de buena calidad.
- Luego, se transfiere de alguna manera el aprendizaje a una nueva tarea, generalmente con un conjunto de datos más limitado, ya sea en cuanto a cantidad, calidad o seguridad.
- Cómo y cuándo son temas de profunda investigación y con gran relevancia en muchos problemas, como por ejemplo el desarrollo de vehículos autónomos, los sistemas de recomendación, asistentes inteligentes y chatbots.



Este tipo de ideas son parte de un área llamada *Transfer Learning*

Source Domain

- Lots of labeled data

$$P_S = (X_S, Y_S)$$



Target Domain

- Limited labels

$$P_T = (X_T, Y_T)$$



Hope is that:

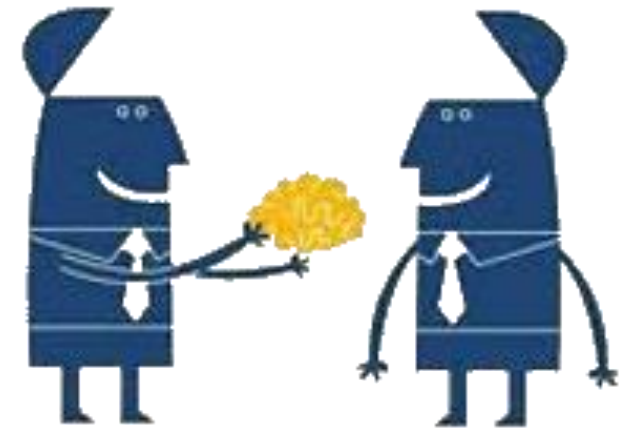
$$P_T \approx P_S$$

Or at least

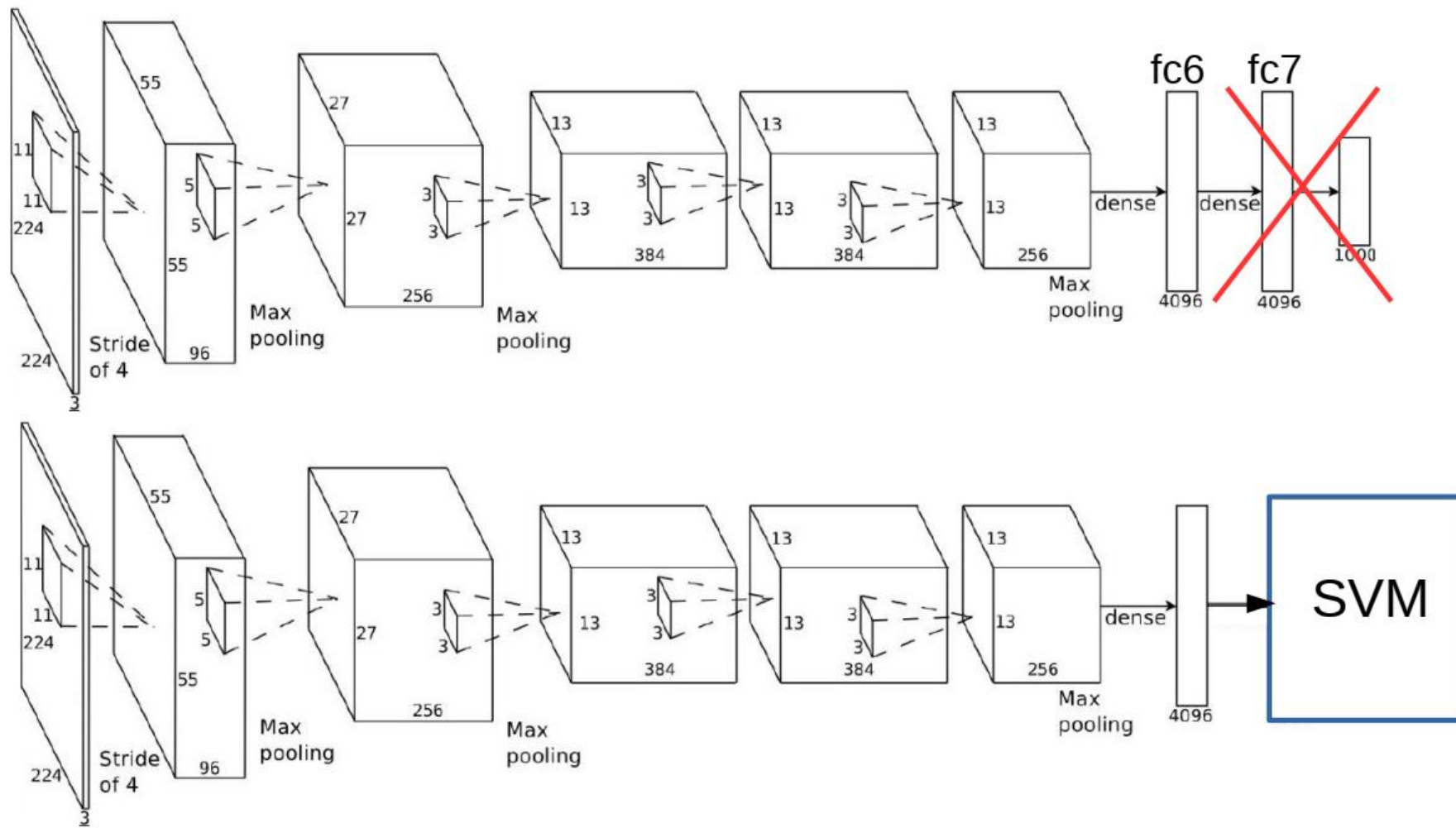
$$P_T(X_T) \approx P_S(X_S)$$

A nivel básico, existen dos mecanismos típicos para hacer *Transfer Learning*

- **Transferencia directa:** *features* aprendidas en un dominio se utilizan directamente en otro, sin noción de la existencia de una red neuronal subyacente.
- **Fine-tuning:** las *features* aprendidas son reentrenadas “suavemente” en el nuevo dominio, cambiando el clasificador/regresor.

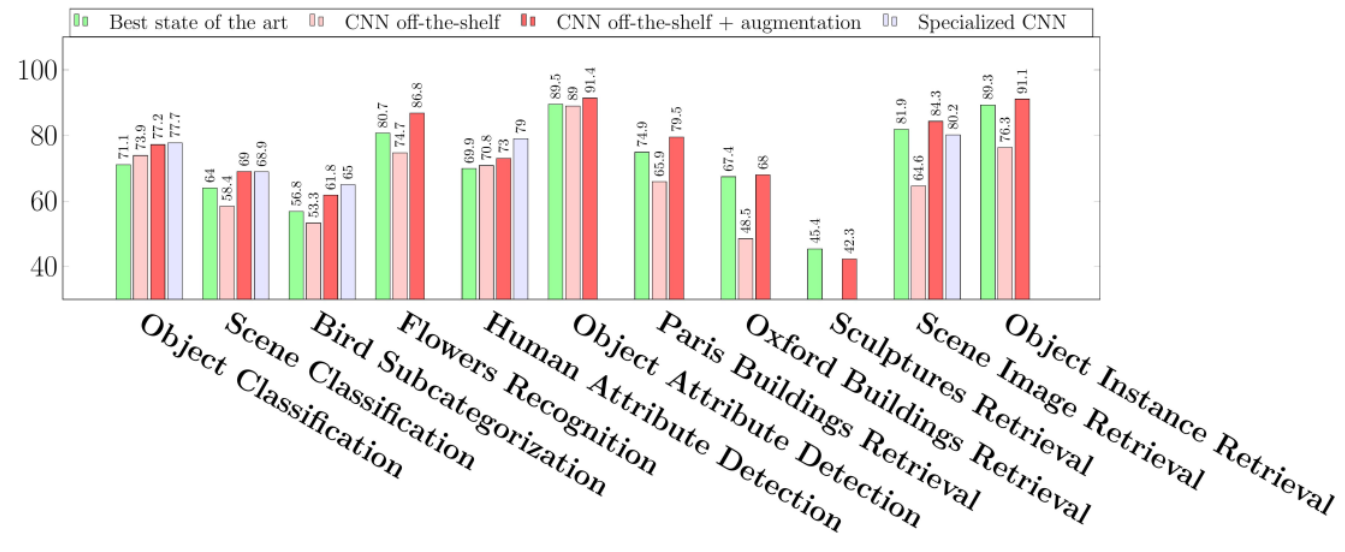
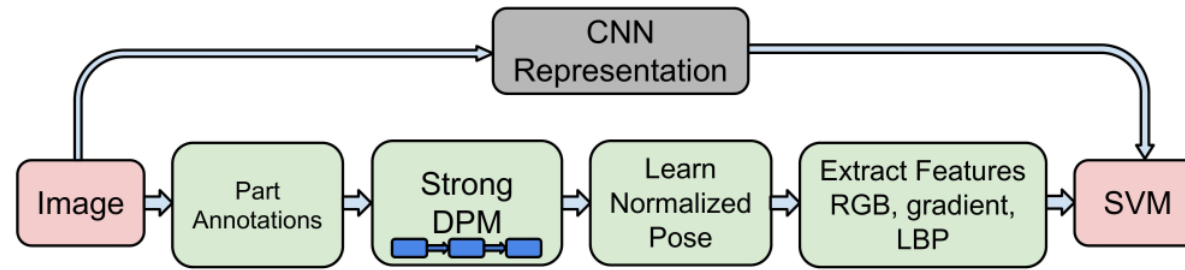


Transferencia directa de features

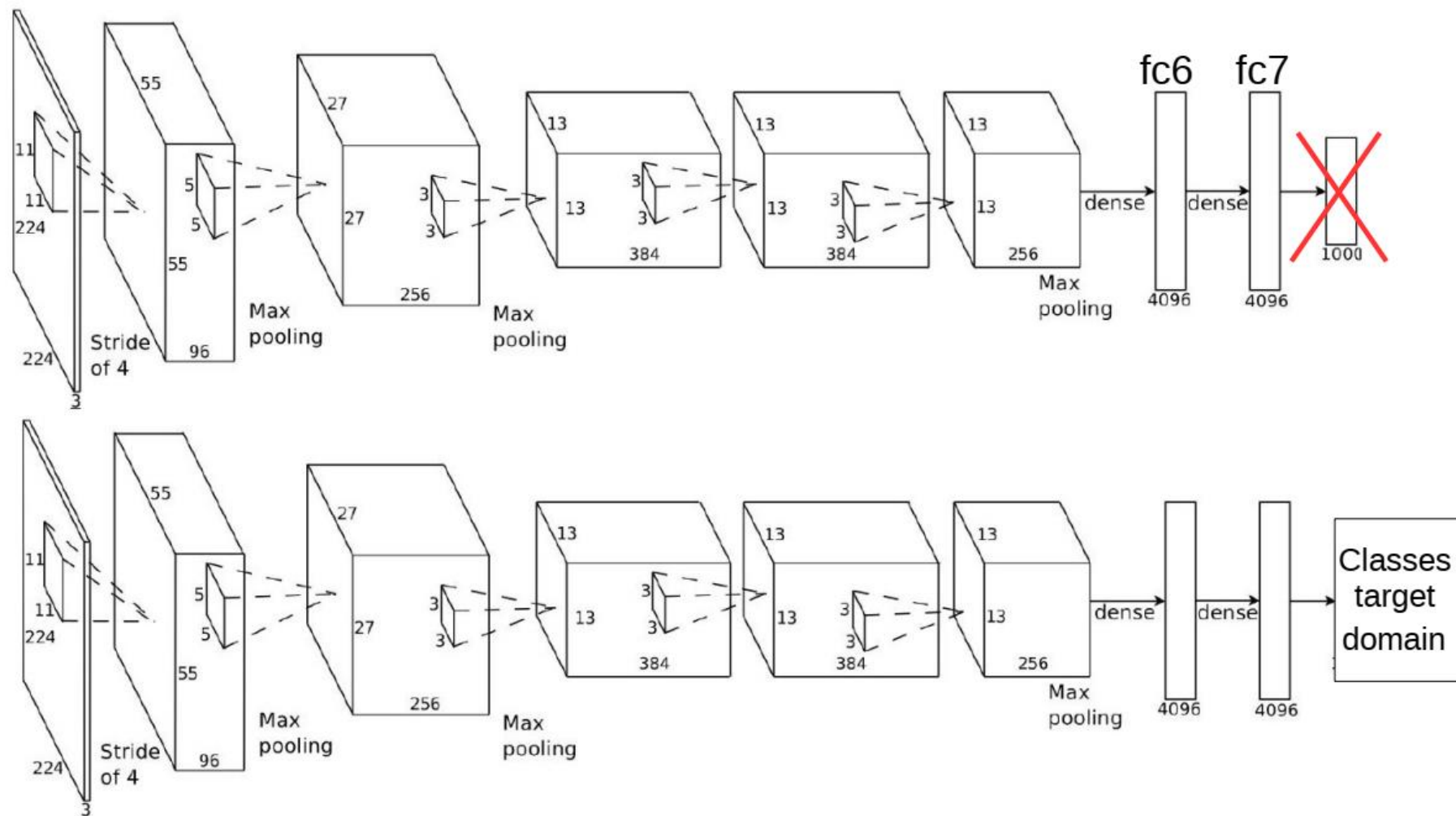


Transferencia directa de features

Ex. A. Razavian et al., “*CNN Features off-the-shelf: an Astounding Baseline for Recognition*”, 2014.

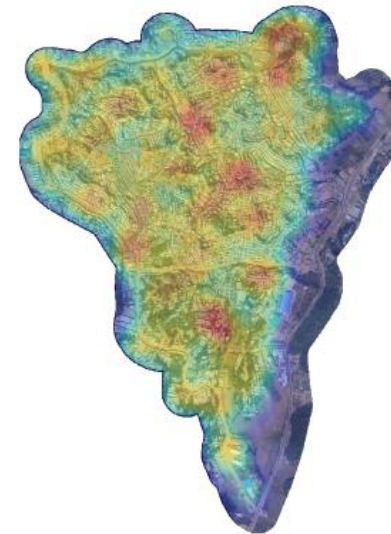
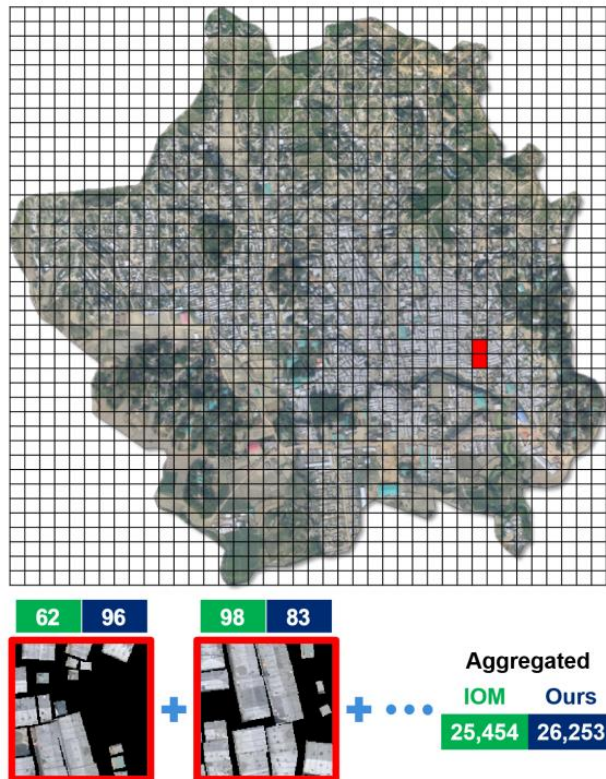


Fine-tuning de *features*

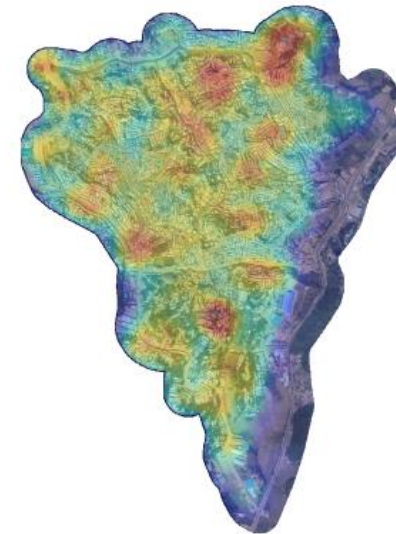


Fine-tuning de *features*

Estimating Displaced Populations From Overhead: Usando una CNN entrenada en ImageNet como base (ResNet-50), se cambia clasificador final por regresor y se reentrena usando imágenes satelitales.



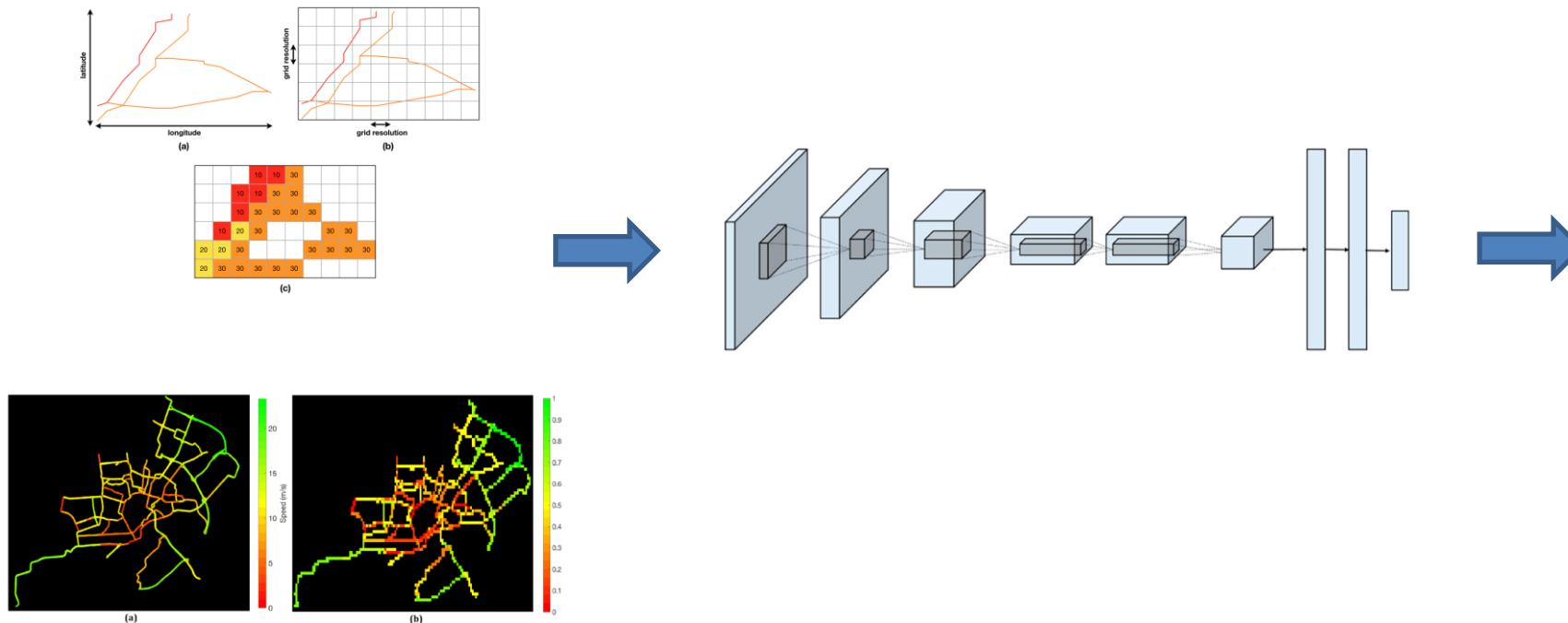
(a) Our Prediction



(b) IOM Estimate

Podemos ir más lejos aún

Understanding Network Traffic States using Transfer Learning: se usa CNN entrenada en ImageNet como base para extraer features de estados de tráfico codificados como imágenes y luego agruparlos.



Class	Medoid	Distribution of time slices in each class	Distribution of days in each class
1			
2			
3			
4			
5			

Transfer Learning presenta una gran cantidad de desafíos

- ¿Reentrenamos todas las capas o algunas? ¿Cuáles? ¿En qué orden?
- ¿Cuándo decidir si se hace *fine-tuning*, transferencia directa o entrenamiento desde cero de una red?
- ¿Cuán similares son los dominios?
- ¿Qué rol juega el *learning rate*?

Vamos a Colab...



Ejercicios propuestos

Parte 1: *Transfer learning*

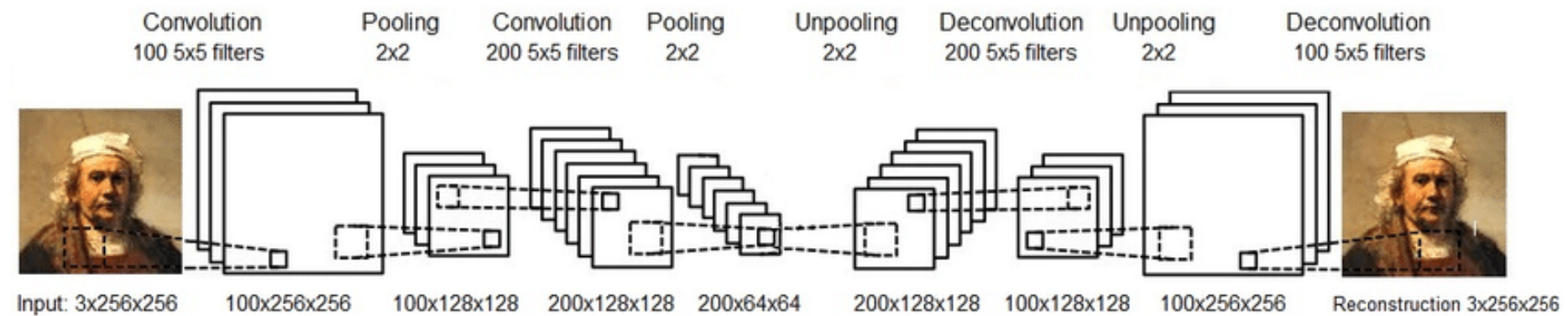
- Entrene una CNN en CIFAR10 o CIFAR100.
- Basándose en el ejemplo, evalúe el rendimiento en transferencia de esta red en el set de datos de gatos y perros.
- Compare el rendimiento obtenido con el de la red entrenada en ImageNet.



Ejercicios propuestos

Parte 2: *Autoencoder*

- Entrene un *Autoencoder* convolucional en CIFAR10 o CIFAR100.
- Basándose en el ejemplo, evalúe el rendimiento en transferencia del *encoder* en el set de datos de gatos y perros.
- Compare el rendimiento obtenido con el de la red entrenada en ImageNet (ejemplo visto en clases) y la de la parte 2.



Entrenar y luego utilizar de forma práctica las redes CNN no es algo trivial

Para estudiar los aspectos prácticos, revisaremos 3 preguntas:

- ¿Cómo entrenar/optimizar?
- ¿Cómo reutilizar lo ya aprendido por una CNN?
- ¿Cómo mejorar las CNNs para llevarlas a entornos reales?

El reconocimiento visual presenta una jerarquía problemas cada vez más complejos

Classification



CAT

No spatial extent

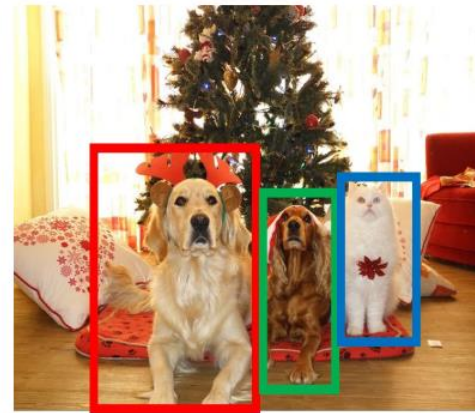
Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



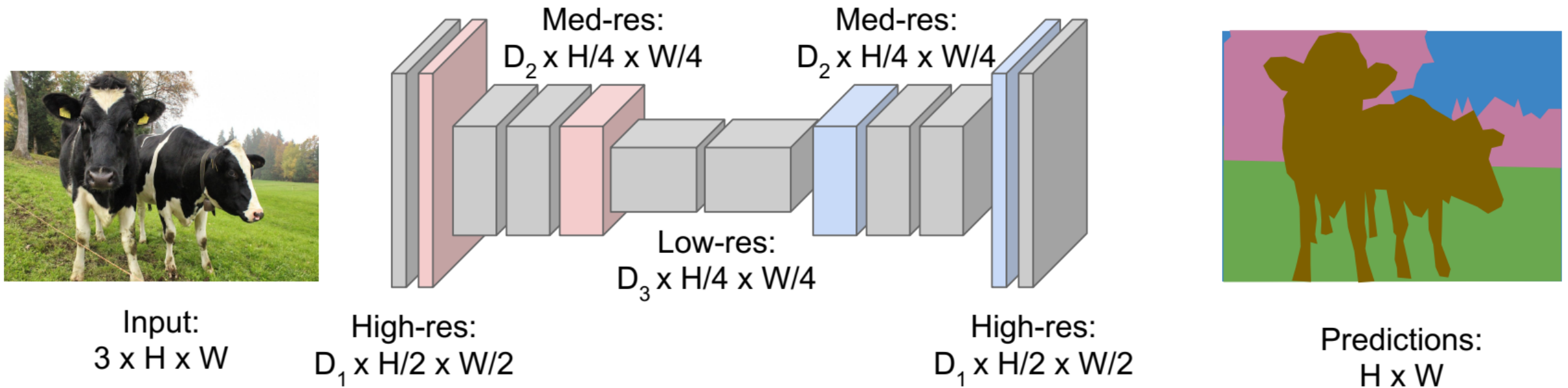
DOG, DOG, CAT

Multiple Object

[This image is CC0 public domain](#)

Para la **segmentación semántica**, se utiliza un esquema *encoder-decoder*

Luego de reducir la dimensionalidad de las features, expandimos el tamaño hasta llegar al original



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

¿Y qué tal los resultados?



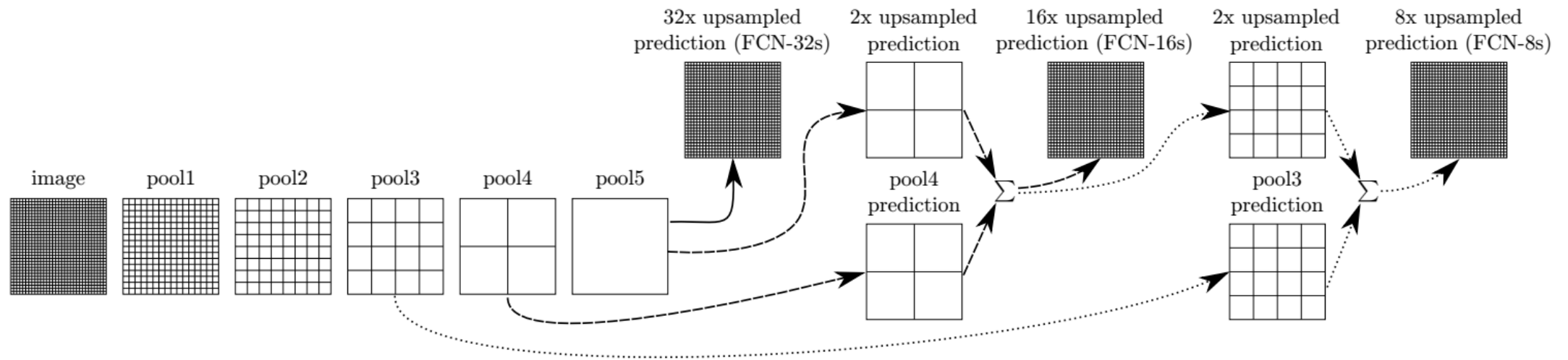
No tan buenos en realidad (bien malos para ser sinceros)

¿Y qué tal los resultados?

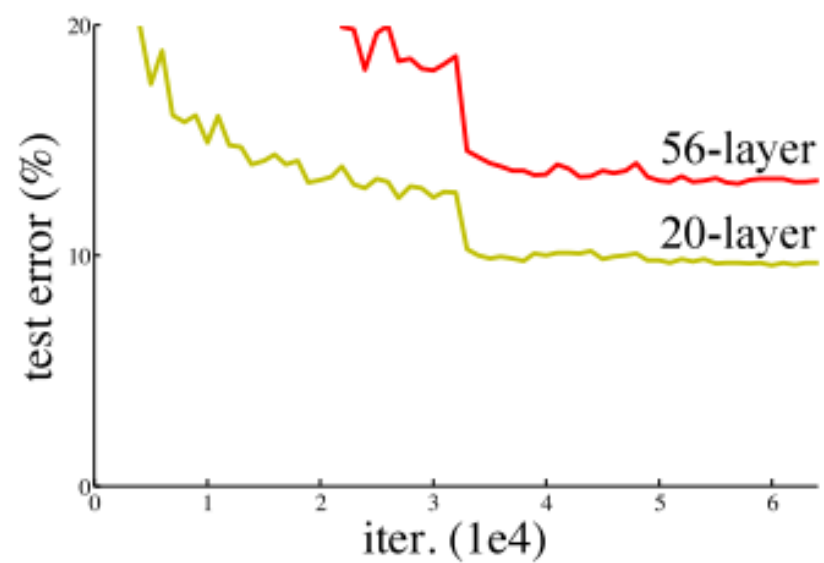
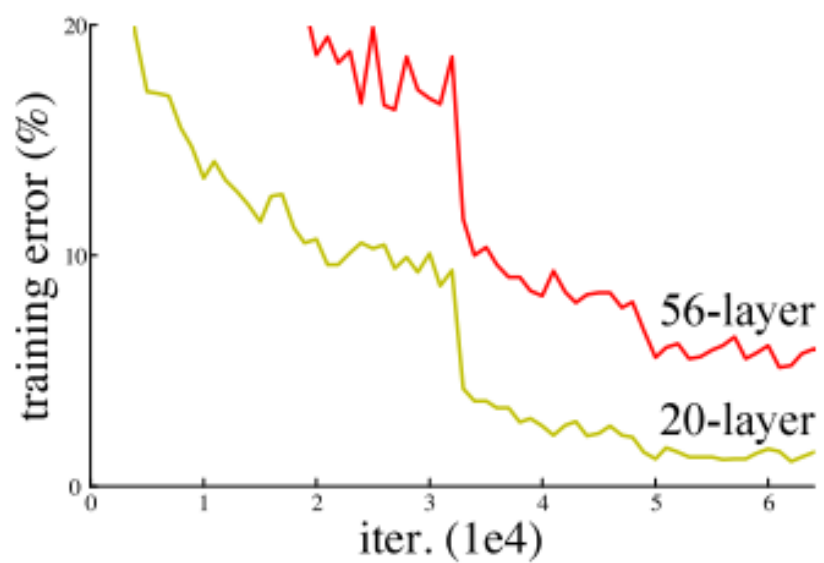


La falta de detalles en los resultados se debe a la pérdida de estos durante el procesamiento

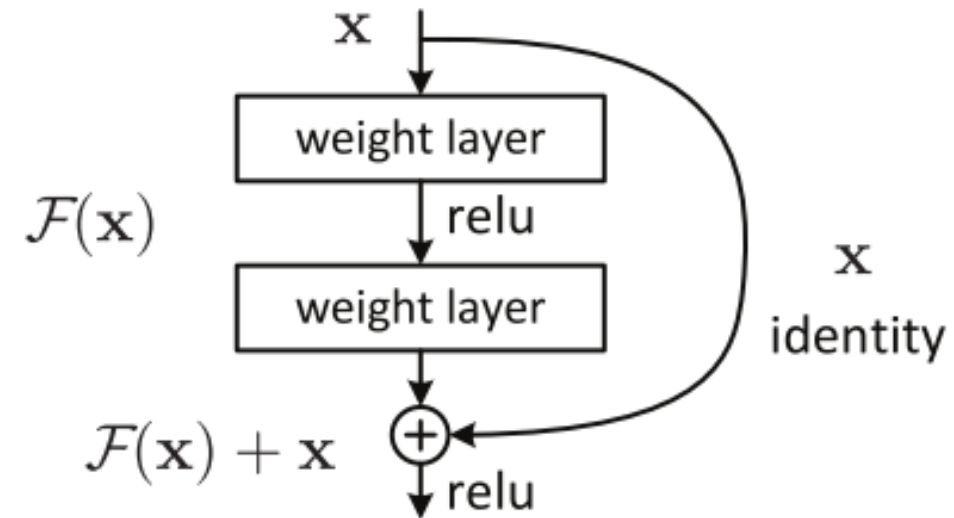
Una forma de atacar el problema es forzando la transferencia directa de detalles, **esquivando** capas



Interludio: *skip connections*



Interludio: *skip connections*



FCN-32s



FCN-16s



FCN-8s



Ground truth



Resultados mejoran mucho, pero el motivo no es claro

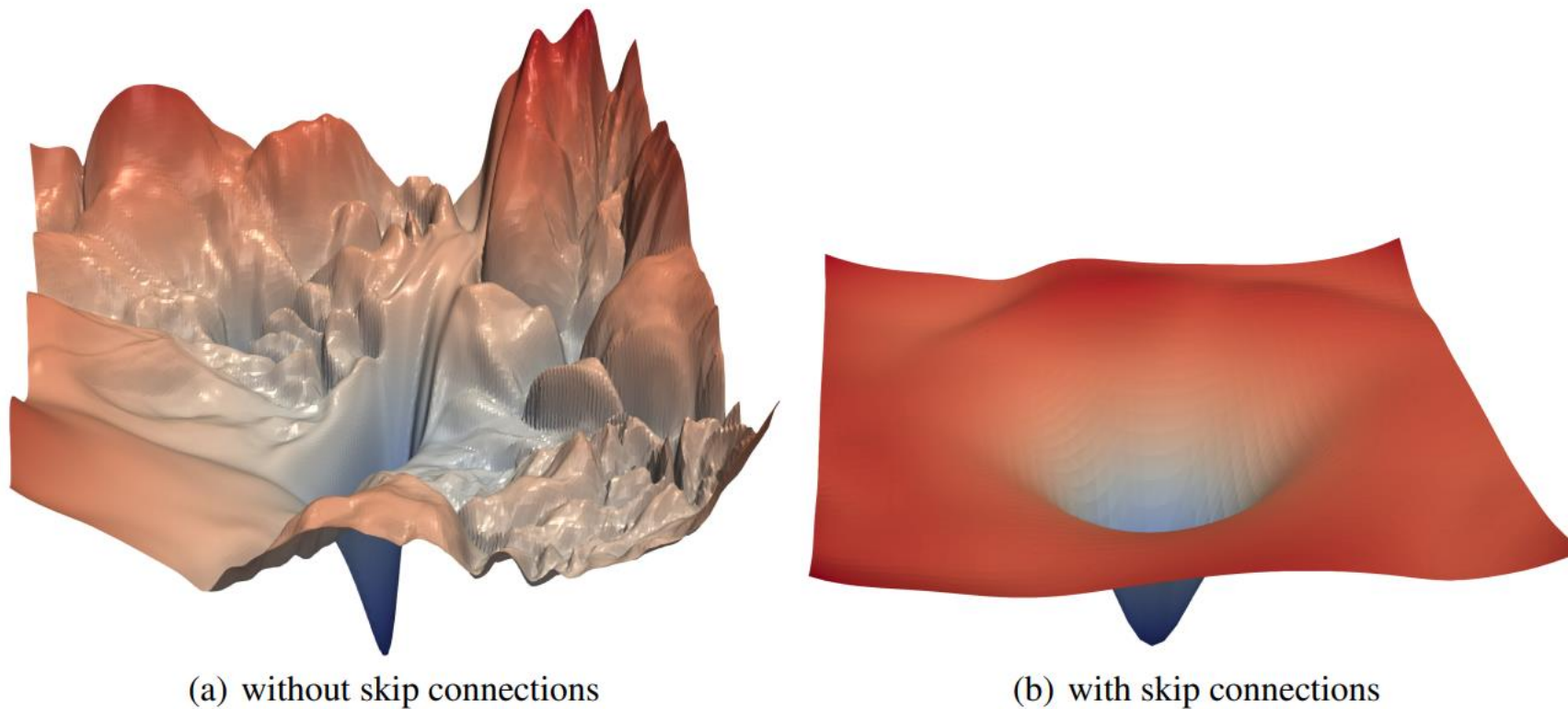


Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Enfoques recientes refinan arquitectura de las redes y de los datos

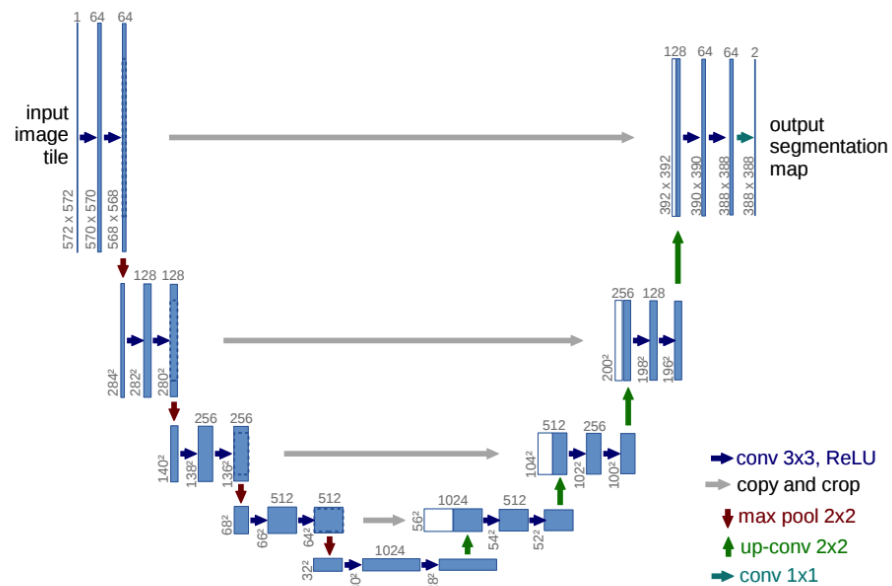
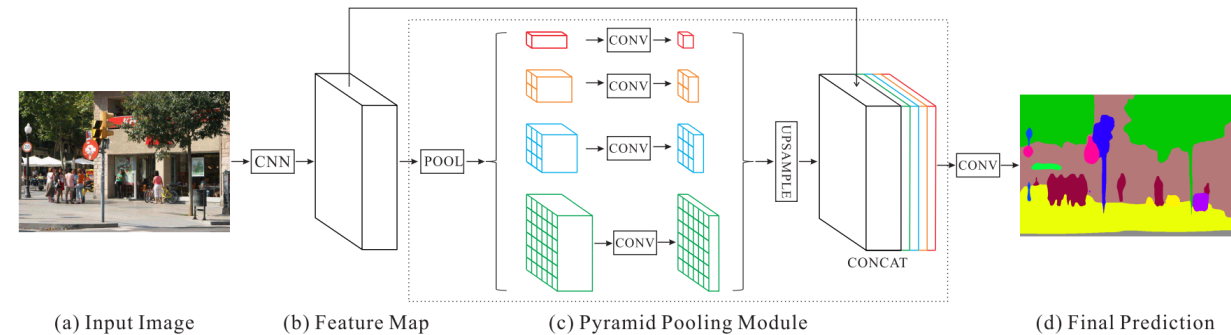


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

Ronneberger, O., Fischer, P., and Brox, T. (2015) "U-net: Convolutional networks for biomedical image segmentation." International Conference on Medical image computing and computer-assisted intervention.



Zha, H., Shi, J., Qi, X., Wang, X., Jia, J. (2017). Multi-Scale Context Aggregation by Dilated Convolutions. Conference on Computer Vision and Pattern Recognition (CVPR)



<https://youtu.be/HYghTzmbv6Q>

Continuemos subiendo en la escala de problemas de visión por computador, hasta la **detección de objetos**

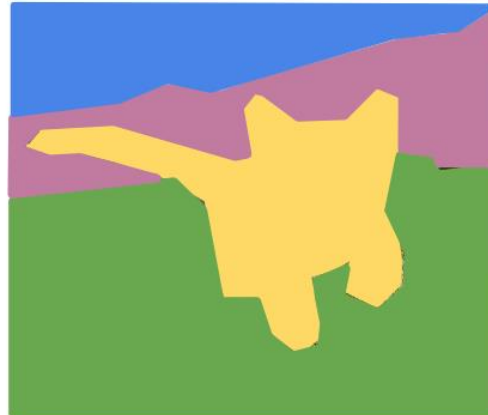
Classification



CAT

No spatial extent

Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

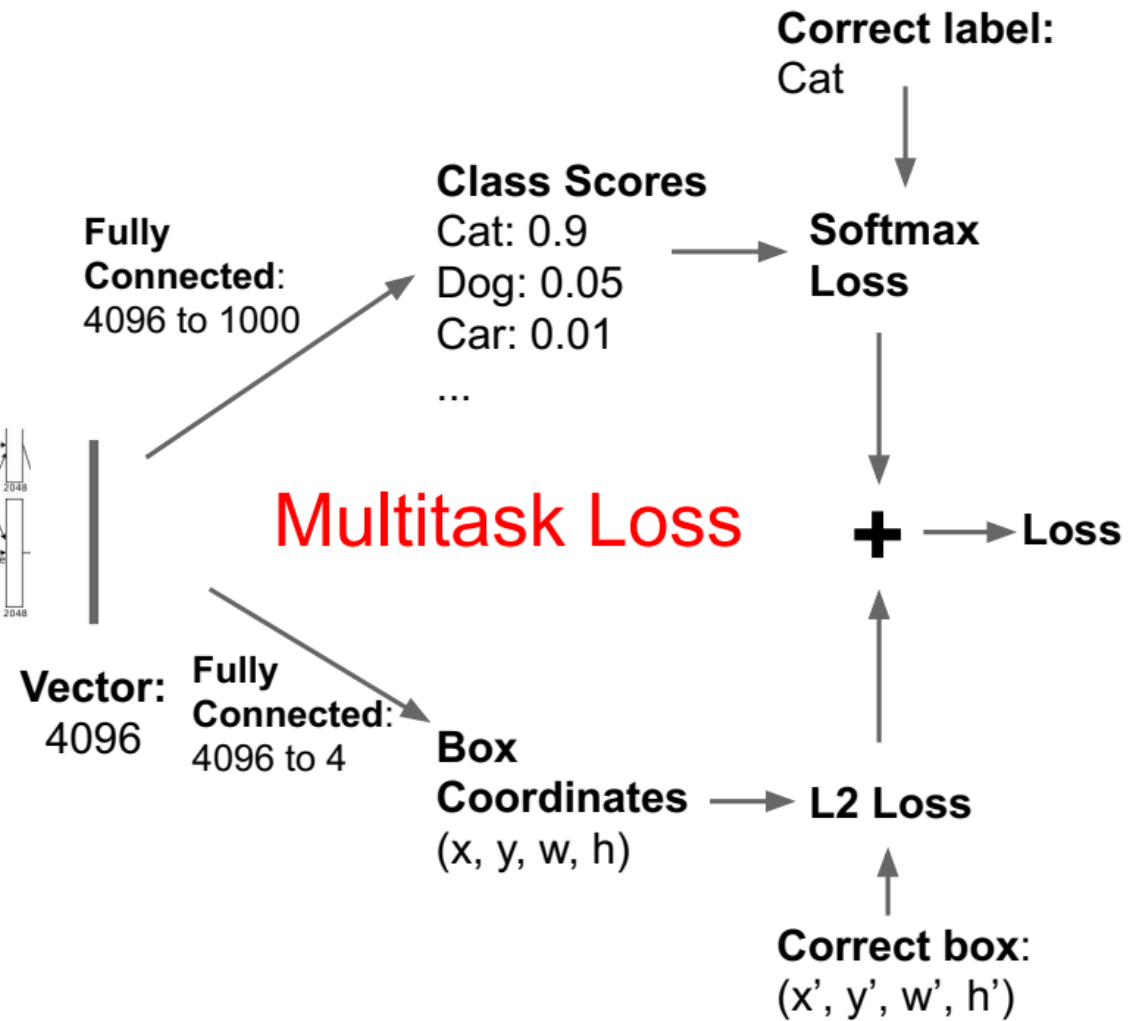
Multiple Object

Instance Segmentation

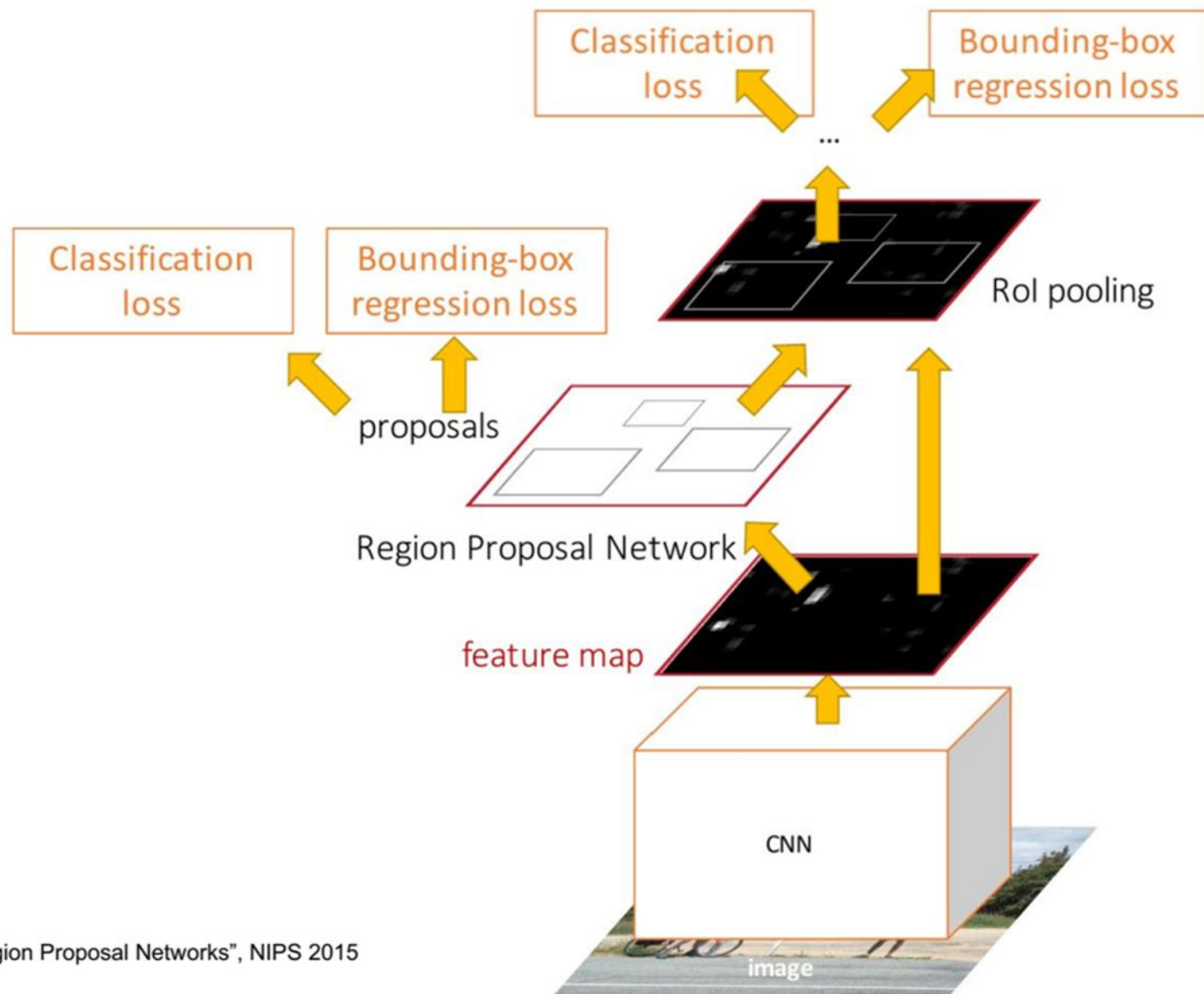


DOG, DOG, CAT

[This image is CC0 public domain](#)



Faster R-CNN:

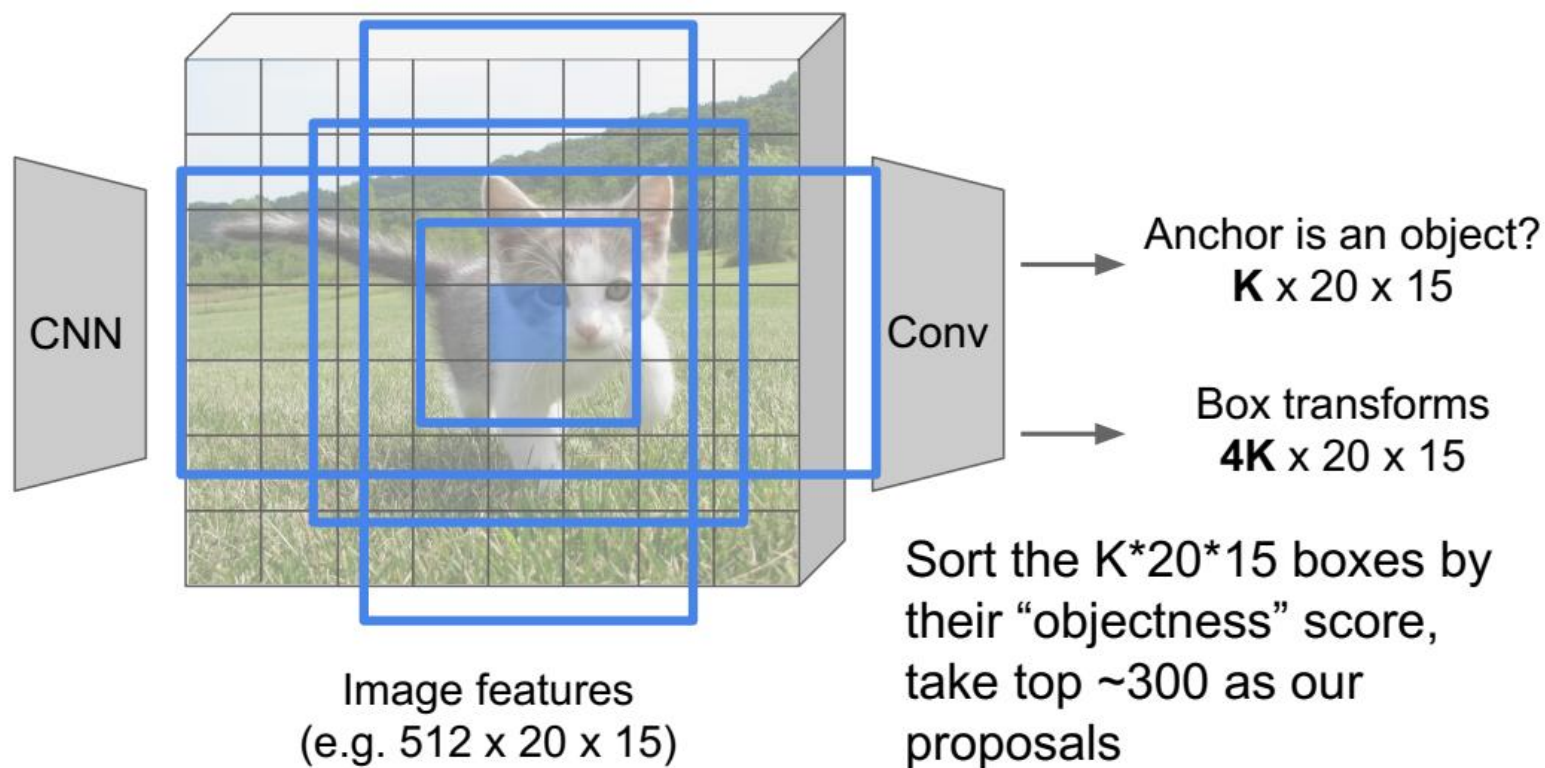


Region Proposal Network

In practice use K different anchor boxes of different size / scale at each point

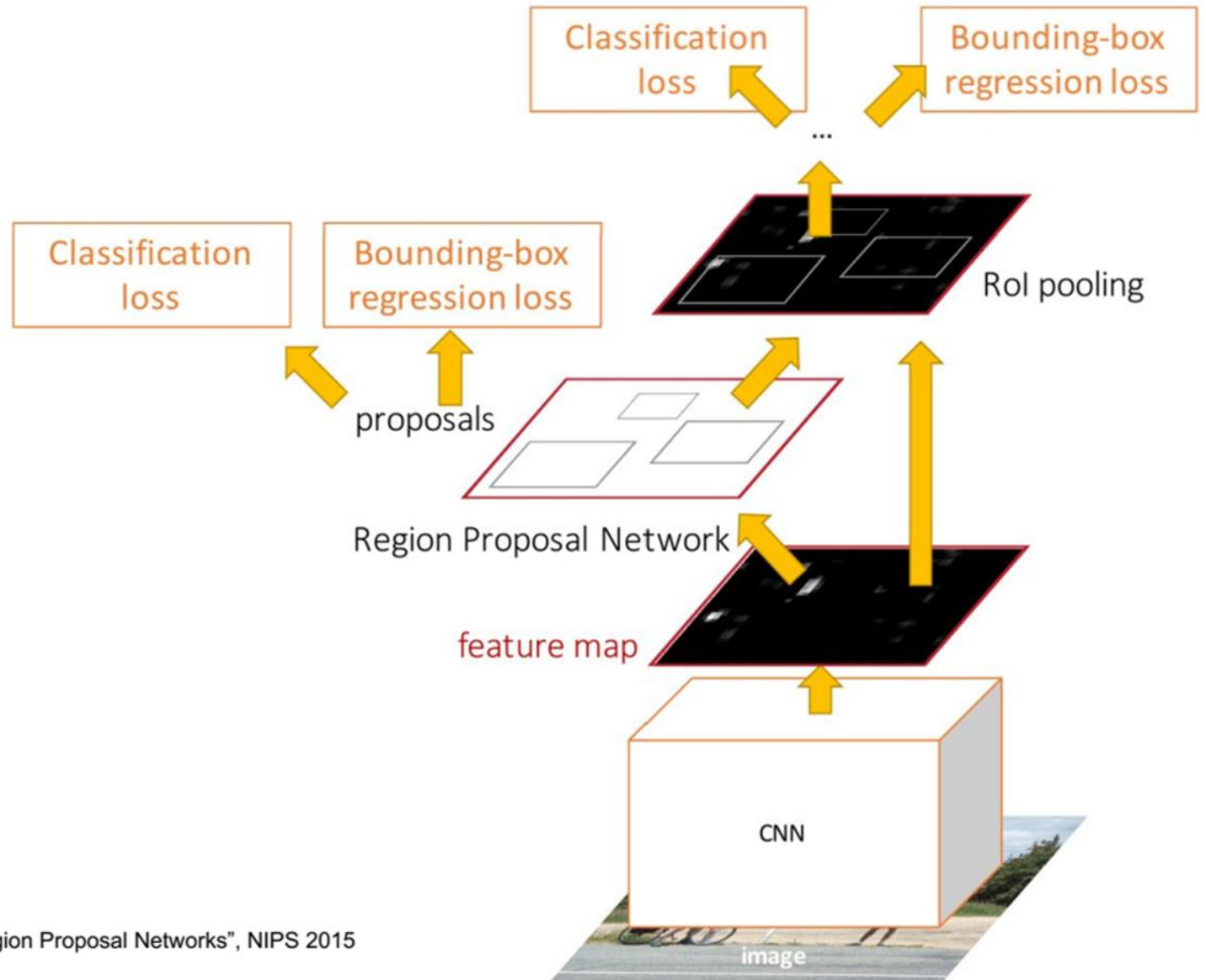


Input Image
(e.g. $3 \times 640 \times 480$)



Faster R-CNN:

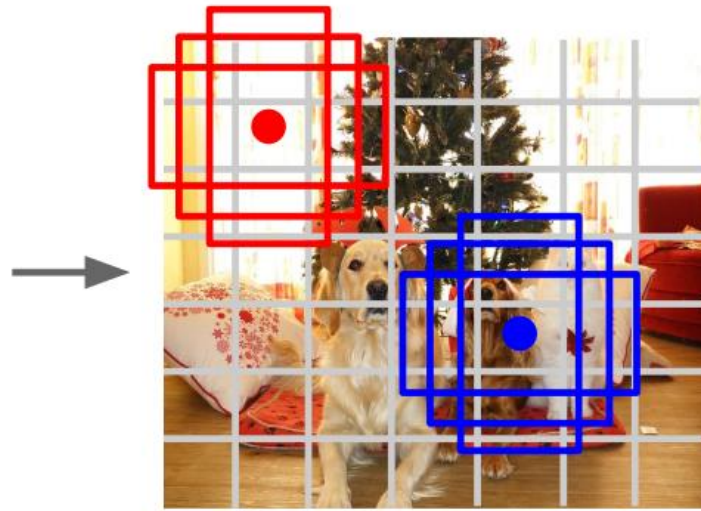
¿Necesitamos realmente la segunda subred?



Single-Stage Object Detectors: YOLO / SSD / RetinaNet



Input image
 $3 \times H \times W$



Divide image into grid
 7×7

Image a set of **base boxes**
centered at each grid cell
Here $B = 3$

Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers:
($dx, dy, dh, dw, \text{confidence}$)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

Output:

$$7 \times 7 \times (5 * B + C)$$

Redmon et al, "You Only Look Once:
Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017



<https://youtu.be/ZyKK4o4HaAM?si=uM4q5L5RpOK9YMg9&t=19>

Para lograr la segmentación a nivel de instancia, utilizamos las mismas ideas

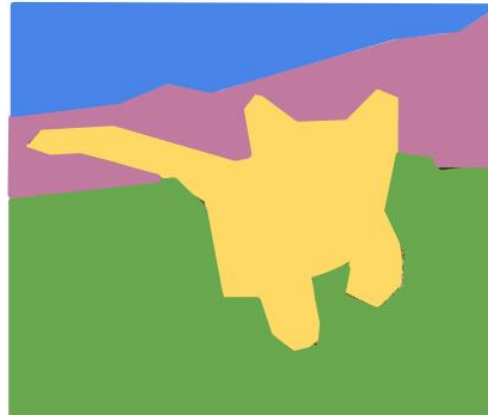
Classification



CAT

No spatial extent

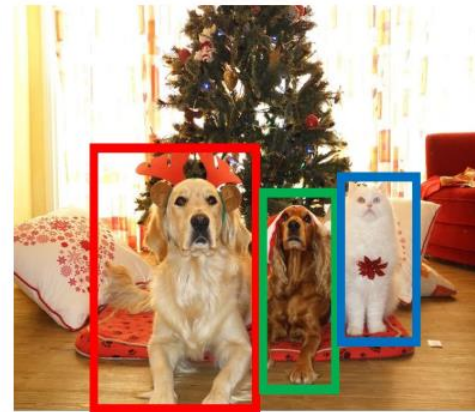
Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

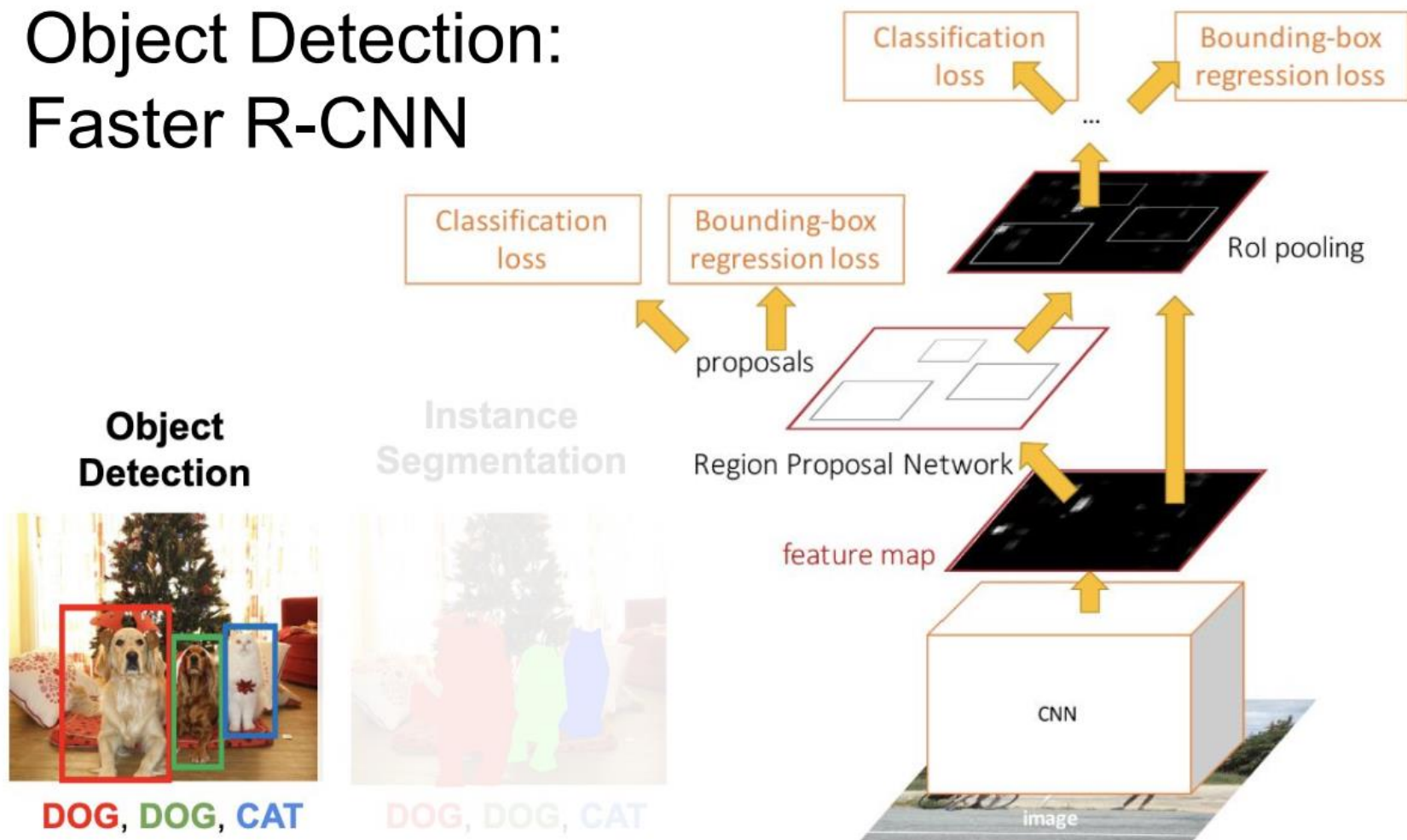
Instance Segmentation



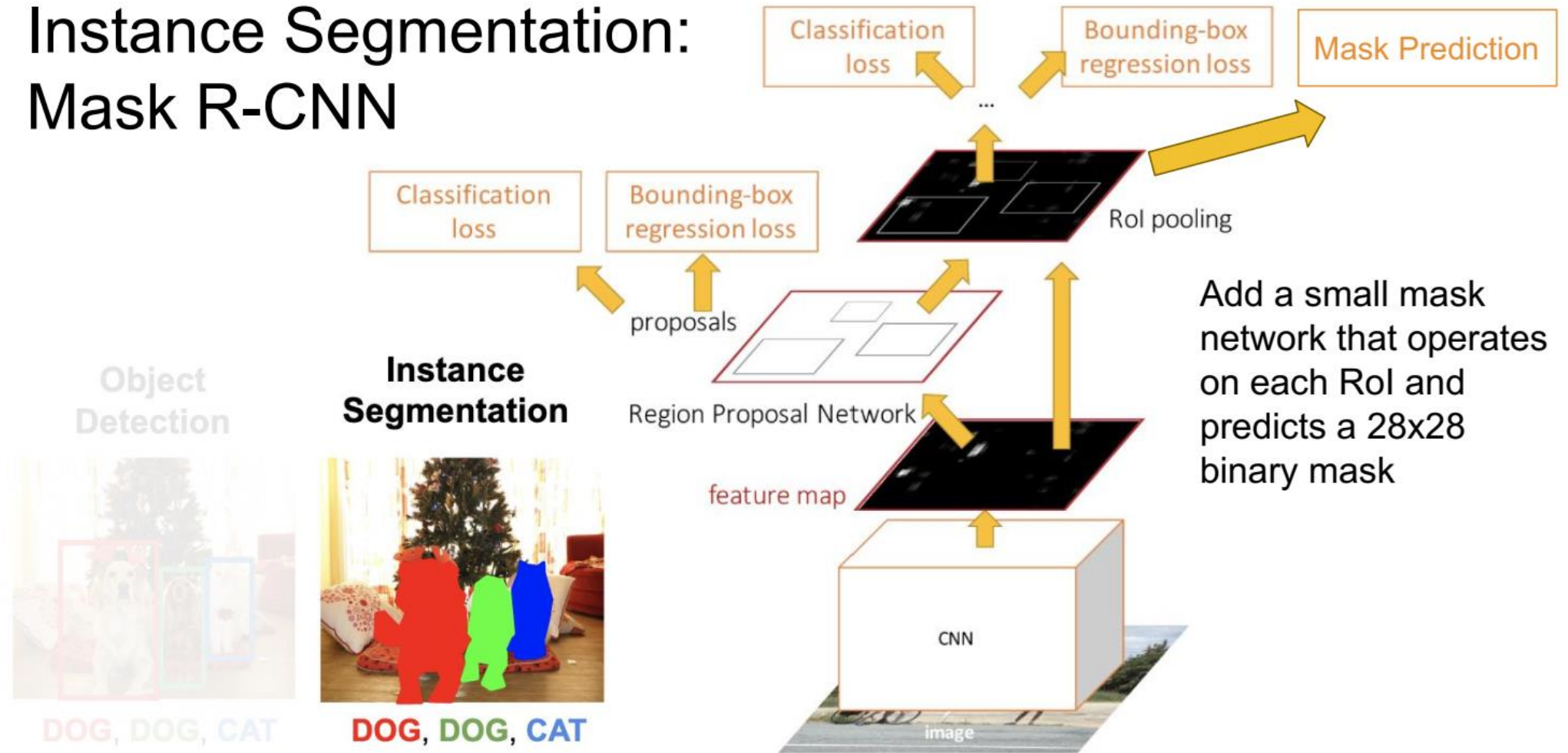
DOG, DOG, CAT

[This image is CC0 public domain](#)

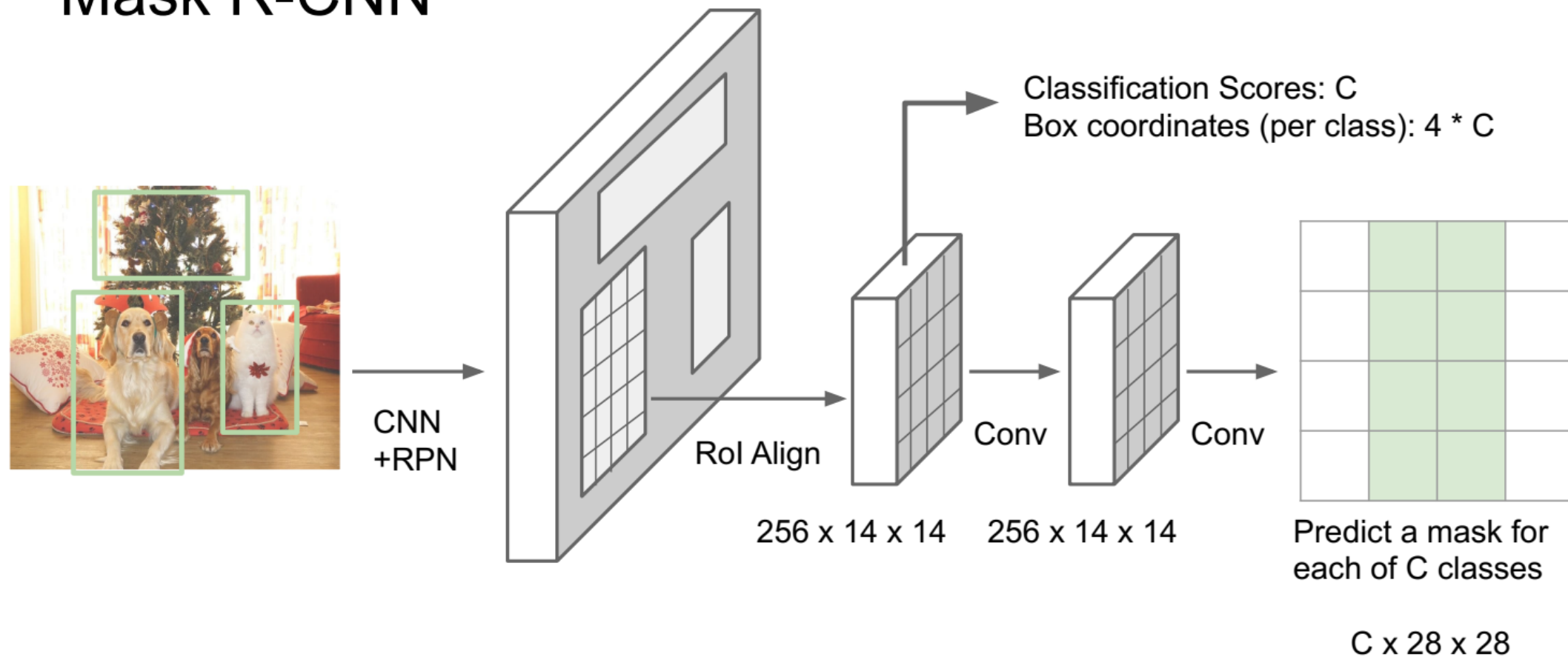
Object Detection: Faster R-CNN

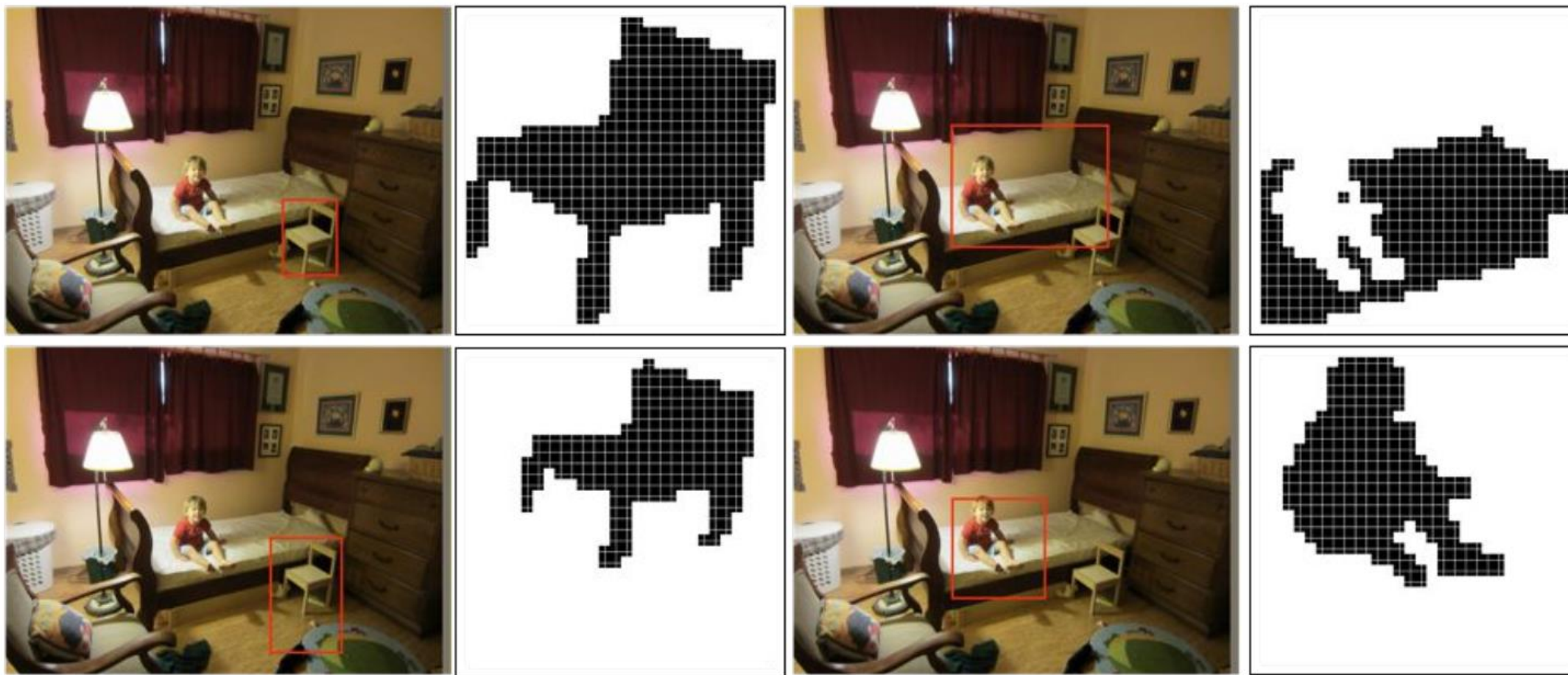


Instance Segmentation: Mask R-CNN

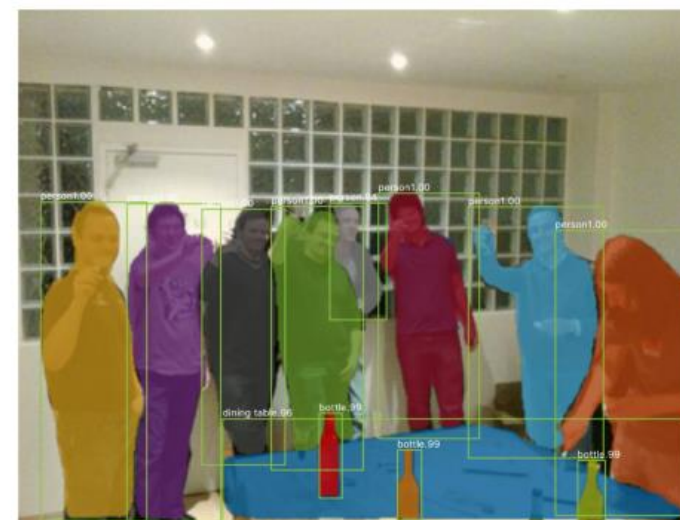
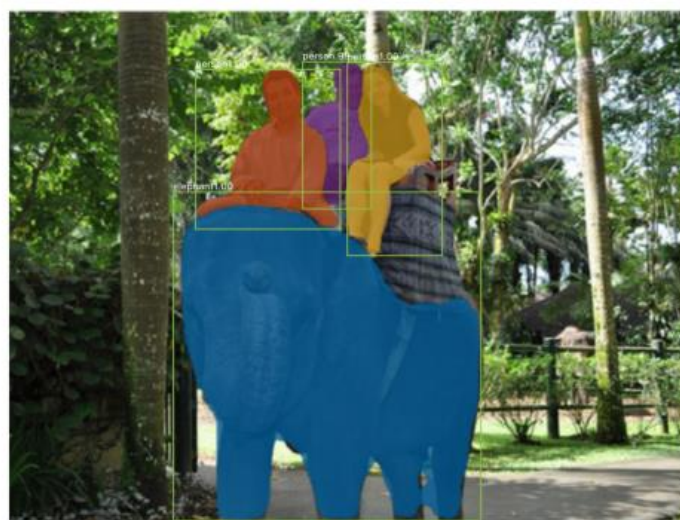


Mask R-CNN

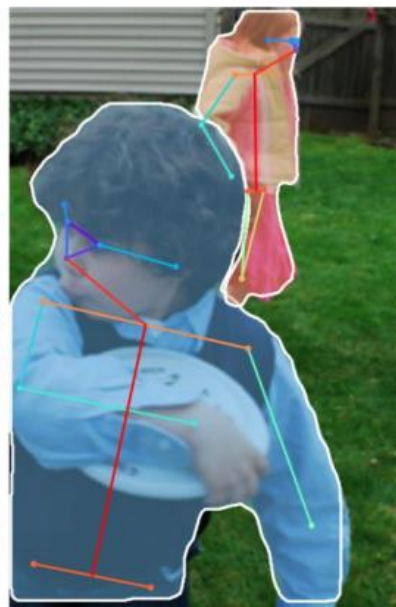
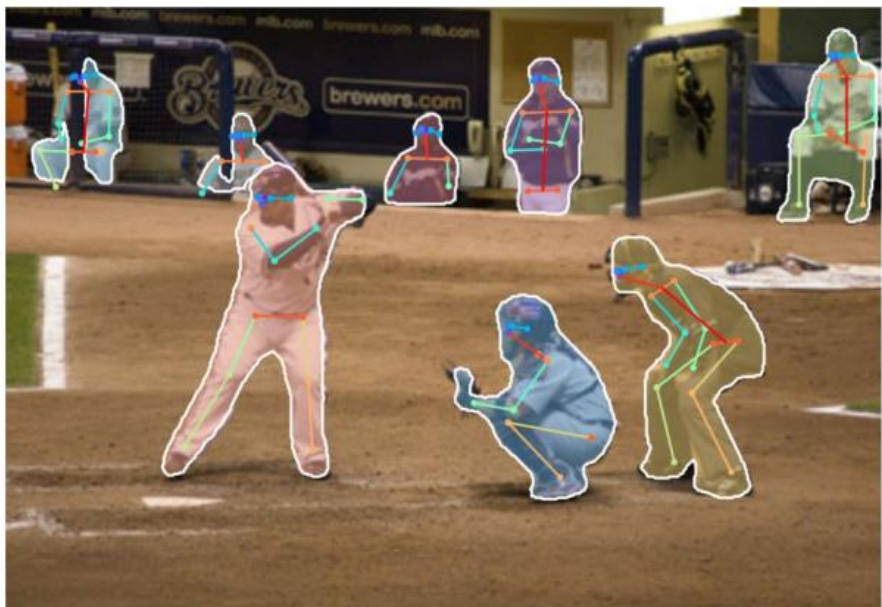




He et al, "Mask R-CNN", ICCV 2017



He et al, "Mask R-CNN", ICCV 2017

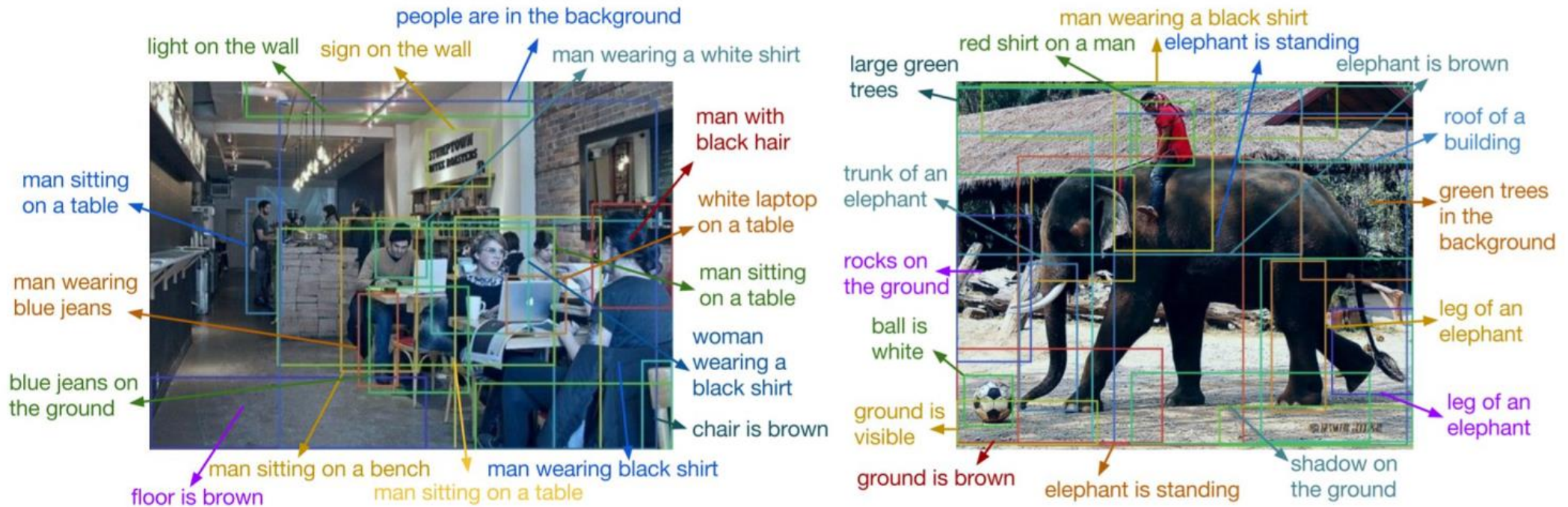


He et al, "Mask R-CNN", ICCV 2017

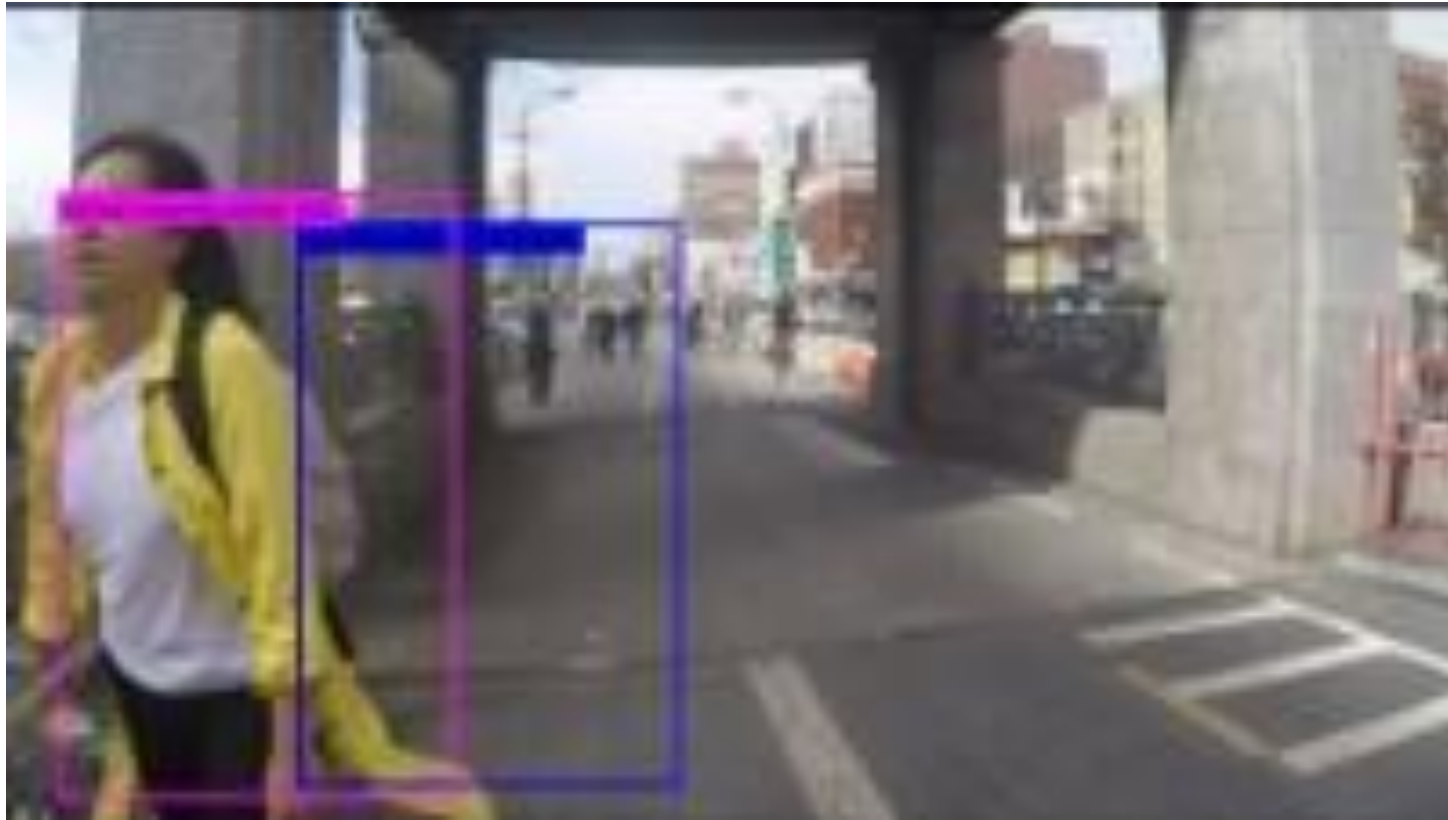


<https://youtu.be/KYNDzlcQMWA>

Object Detection + Captioning = Dense Captioning



Johnson, Karpathy, and Fei-Fei, "DenseCap: Fully Convolutional Localization Networks for Dense Captioning", CVPR 2016
Figure copyright IEEE, 2016. Reproduced for educational purposes.



<https://youtu.be/25jyl67-poQ>

Algunos comentarios al cierre

- Para entrenar redes hay que considerar muchos elementos prácticos
- Afortunadamente, no es necesario reinventar la rueda y muchas veces es suficiente basarse trabajos anteriores.
- CNNs pueden adaptarse para enfrentar diversos problemas de reconocimiento visual.
- Modificaciones arquitectónicas y combinaciones de redes son los principales enfoques.
- Solución de múltiples tareas simultáneas permite avanzar a la comprensión de imágenes (*image understanding*).

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



Deep Learning

Aspectos prácticos de CNN

Ariel Reyes

Dpto. Ciencia de la Computación